



AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH (AIUB)

Course: Advanced Database Management System

Semester: Summer 2025-2026

Section: C

PROGRAMMING CONTEST MANAGEMENT SYSTEM

Faculty: Juena Ahmed Noshin

Project Group- 05

Name	Student ID	Contribution Percentage	Contribution Topics
ABU NAYEM MD ARMAN	22-47249-1	25%	Planning, Introduction, Use Case Diagram, Activity Diagram
SHISHIR MAHMUD	22-47683-2	25%	Project Proposal, Class Diagram, Table Creation SQL, Data Insertion SQL, Conclusion
MOHAMMAD ASAD BIN JAFOR	23-50088-1	25%	Normalization, Schema Diagram, Basic PL/SQL, Documentation
ROMAN SHEIKH RAZAN	23-50460-1	25%	Scenario Description, ER Diagram, UI Planning, Advanced PL/SQL

Contents

Section	Page
1. Introduction	3
2. Project Proposal	3
3. Class Diagram	4
4. Use Case Diagram	5
5. Activity Diagram	6
6. User Interface Planning	11
7. Scenario Description	18
8. ER Diagram	19
9. Normalization up to 3NF	20
10. Schema Diagram	24
11. Table Creation Using SQL	25
12. Data Insertion Using SQL	31
13. Query Writing Using PL/SQL	37
13.1 Basic PL/SQL	37
13.2 Advanced PL/SQL	52
14. Conclusion	73

1. Introduction

The Programming Contest Management System is designed to simplify the process of organizing, managing, and participating in online programming contests. It provides a centralized platform where organizers can create and manage contests, add programming problems and test cases, publish announcements, answer clarification questions, and monitor participant activities.

Participants can join contests, solve programming problems, submit their solutions using different programming languages, ask clarification questions, and view their performance through the leaderboard. The system stores important information about users, contests, problems, test cases, submissions, announcements, clarifications, and rankings in an organized manner.

By bringing all contest-related activities into one platform, the system reduces manual work, improves accuracy, supports fair evaluation, and helps organizers and participants manage programming contests more efficiently.

2. Project Proposal

We propose the development of the Programming Contest Management System (PCMS). It is a relational database system for managing online programming contests. The system will support the complete contest process. This includes user registration, contest creation, problem management, test case management, solution submission, evaluation, clarification, announcements, and leaderboard generation. All important information will be stored in one organized database.

Problem Statement

Programming contests are often managed using spreadsheets, documents, emails, and separate online tools. This may work for small contests. However, it becomes difficult when many participants, problems, submissions, and contest activities are involved. Information may be duplicated, missing, or incorrect. Organizers may also struggle to manage contest schedules, problems, test cases, participant submissions, announcements, and clarification requests. Evaluating a large number of submissions manually can be slow and inaccurate. Participants may also face difficulty checking submission results and current rankings. A single and reliable system is needed to manage the complete programming contest process.

Proposed Solution

PCMS will store all programming contest information in one organized database. Organizers will be able to create contests, add problems and test cases, publish announcements, and answer clarification questions. Participants will be able to register, view contests, solve problems, and submit solutions using different programming languages.

The system will record submission details such as source code, verdict, execution time, memory usage, and submission date. It will also manage leaderboard records, including solved problem count, penalty, and rank position. Important rules will be handled through SQL and PL/SQL programs to ensure accurate contest management.

The final outcome will be a working database system for managing a programming contest from beginning to end.

3. Class Diagram

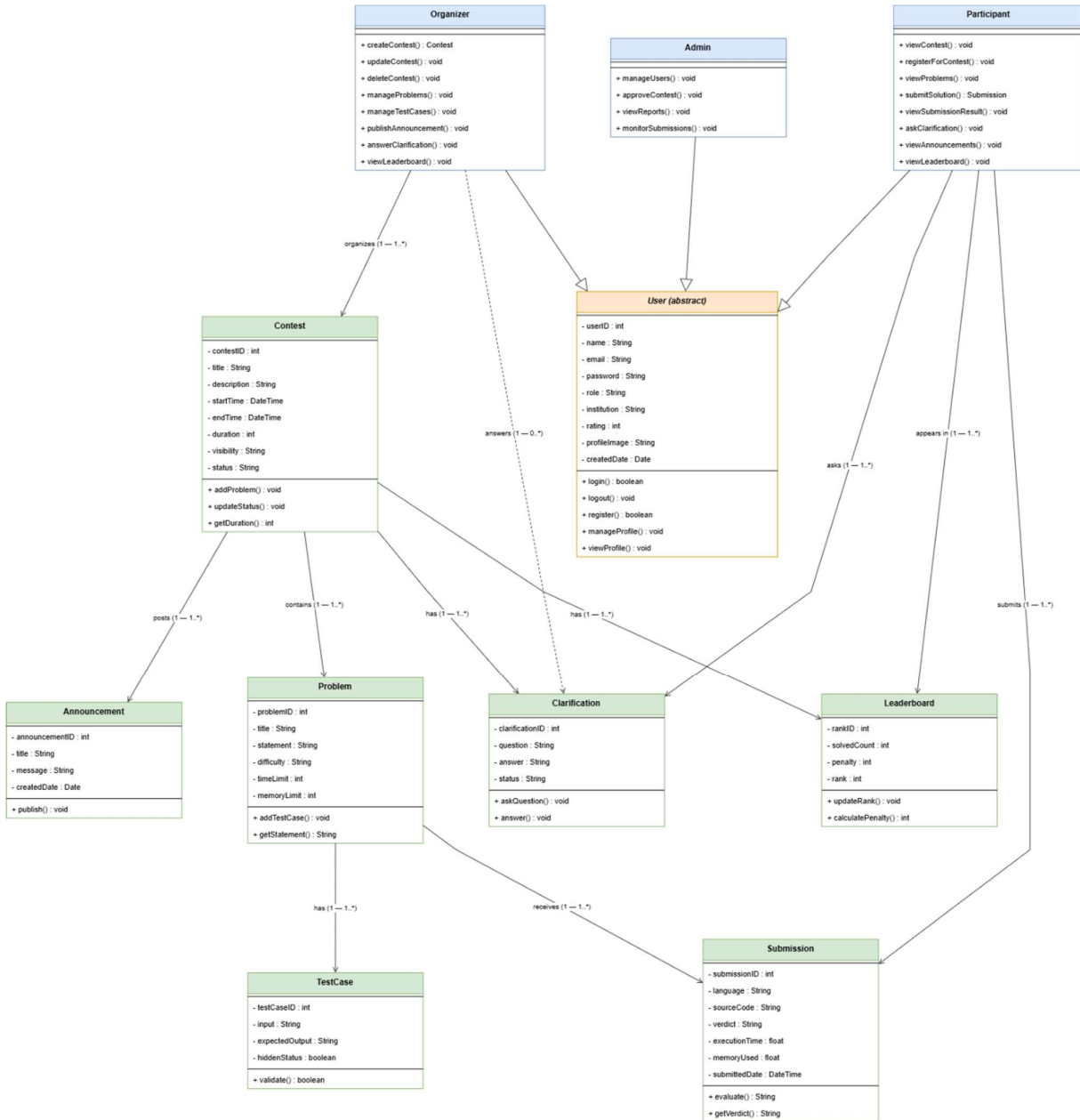


Figure 1: Class Diagram of the Programming Contest Management System

4. Use Case Diagram

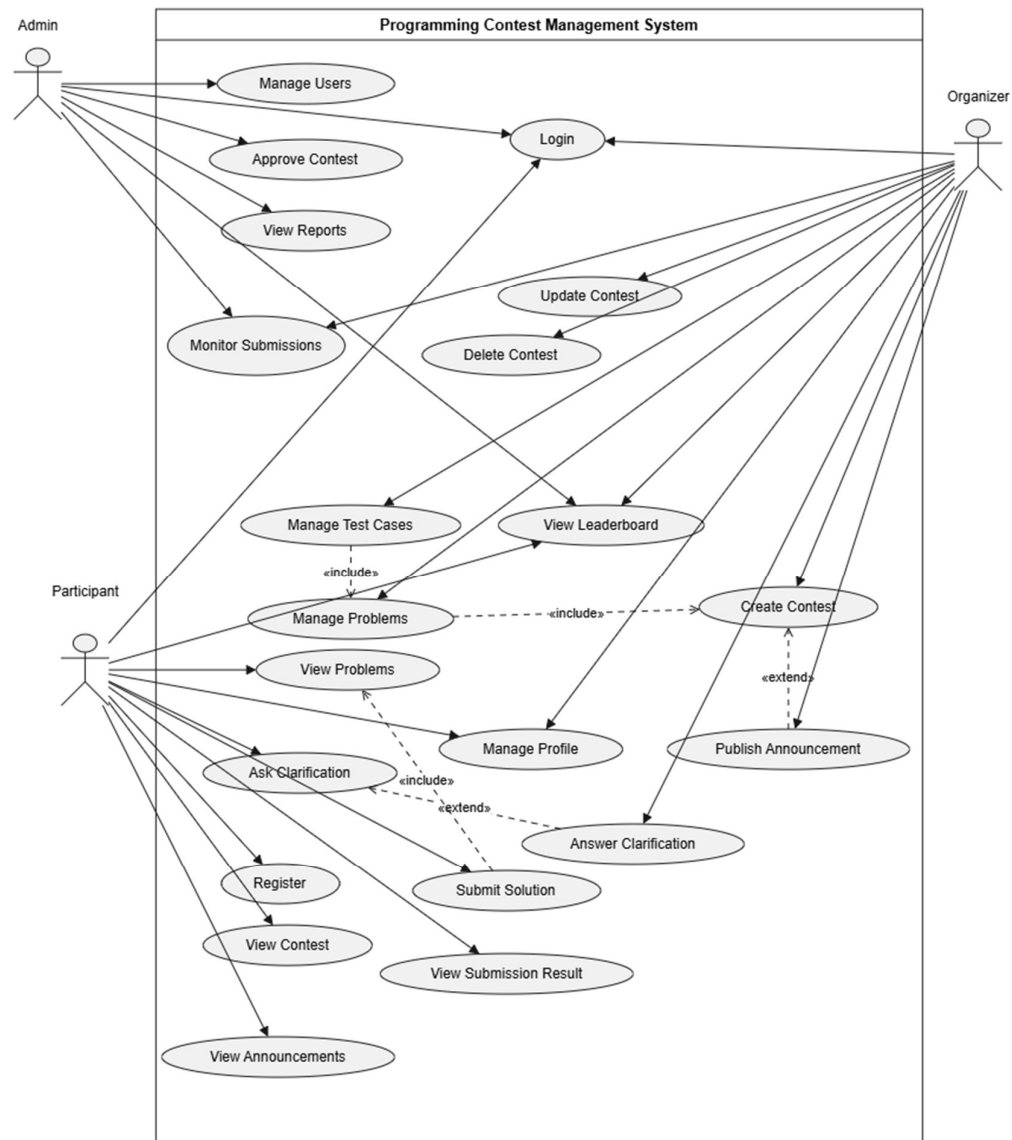


Figure 2: Use Case Diagram

5. Activity Diagram

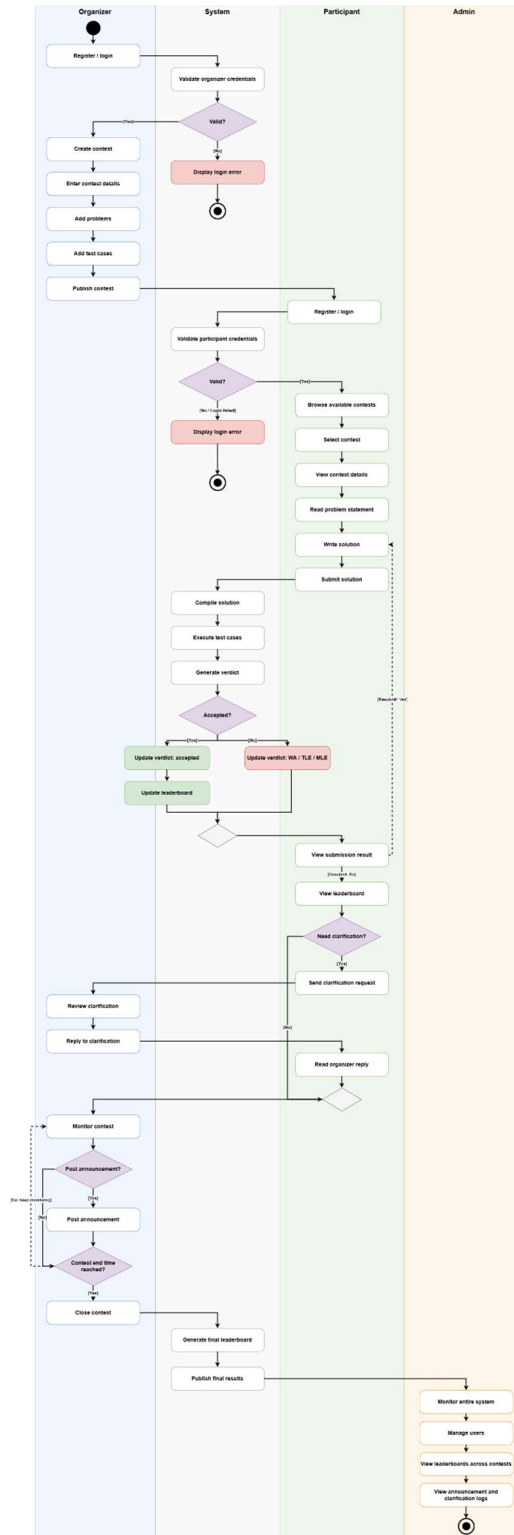
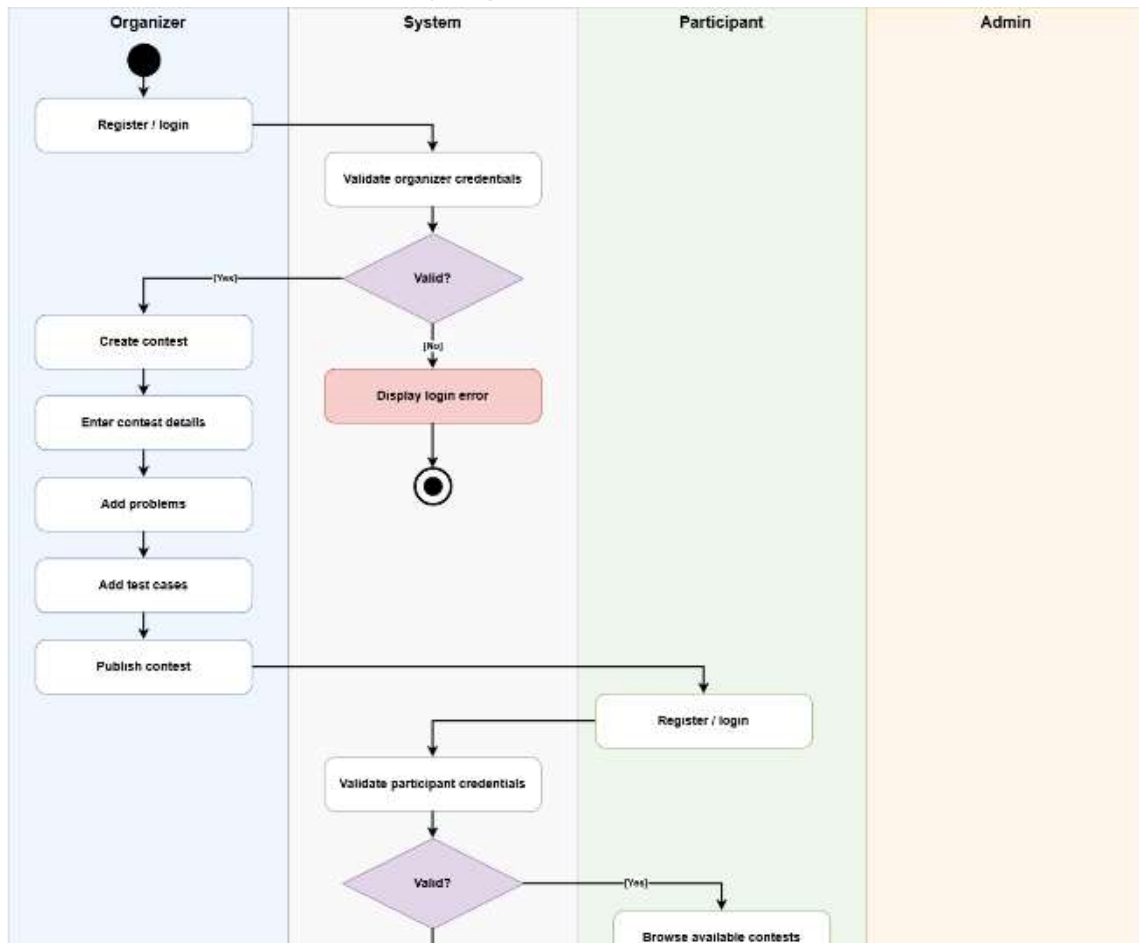
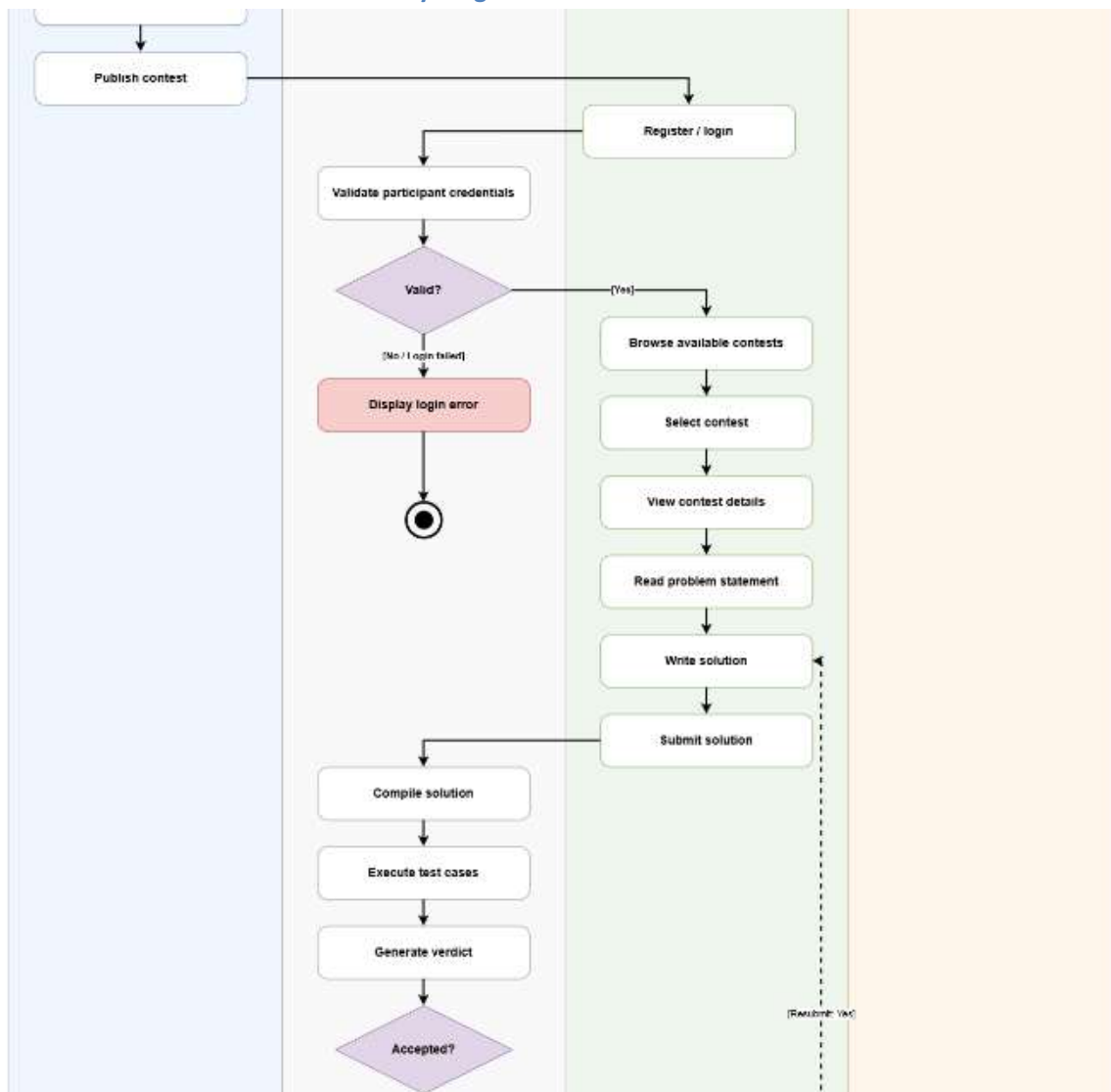


Figure 3: Complete Activity Diagram

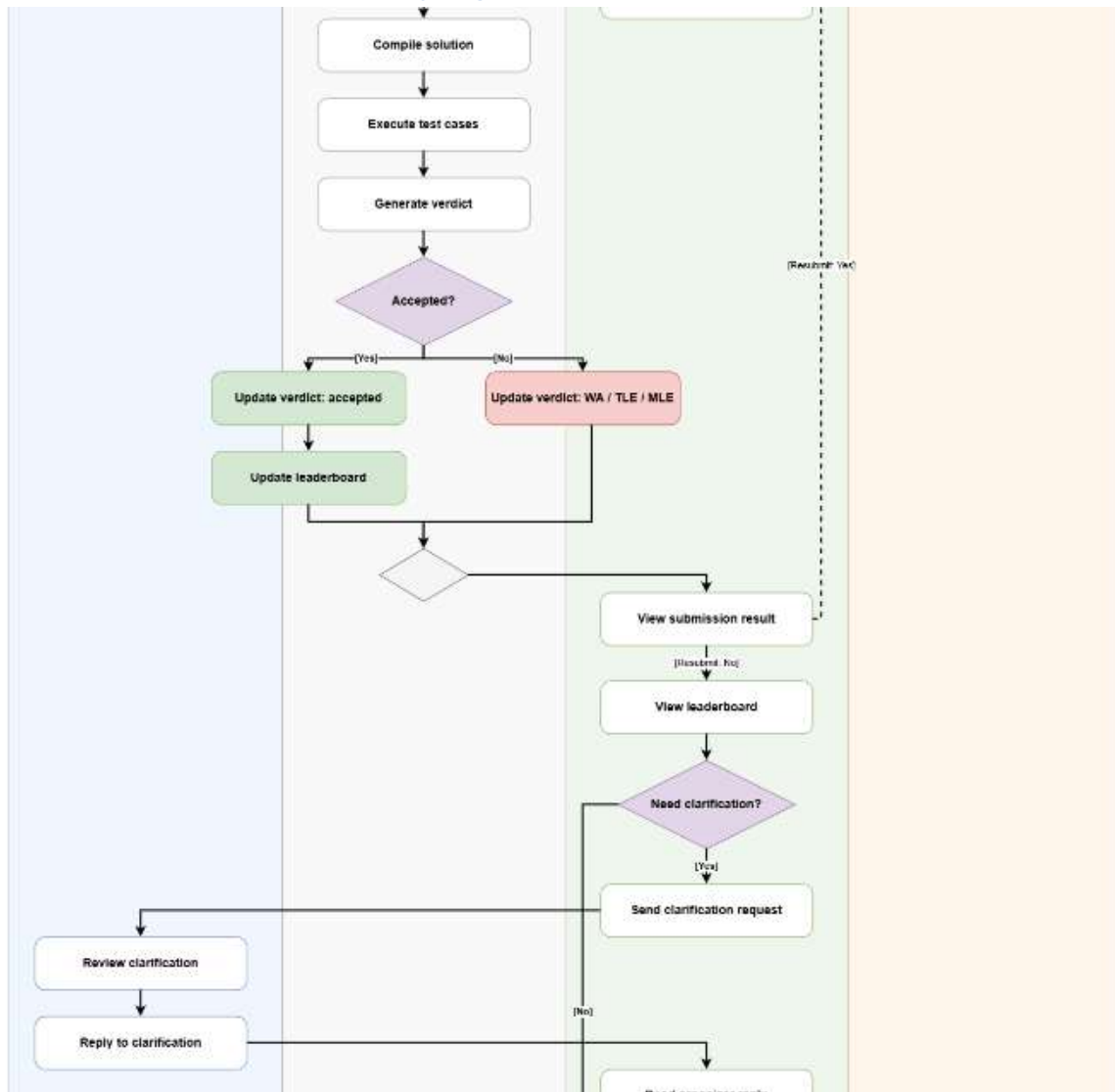
Activity Diagram - Detailed View 1



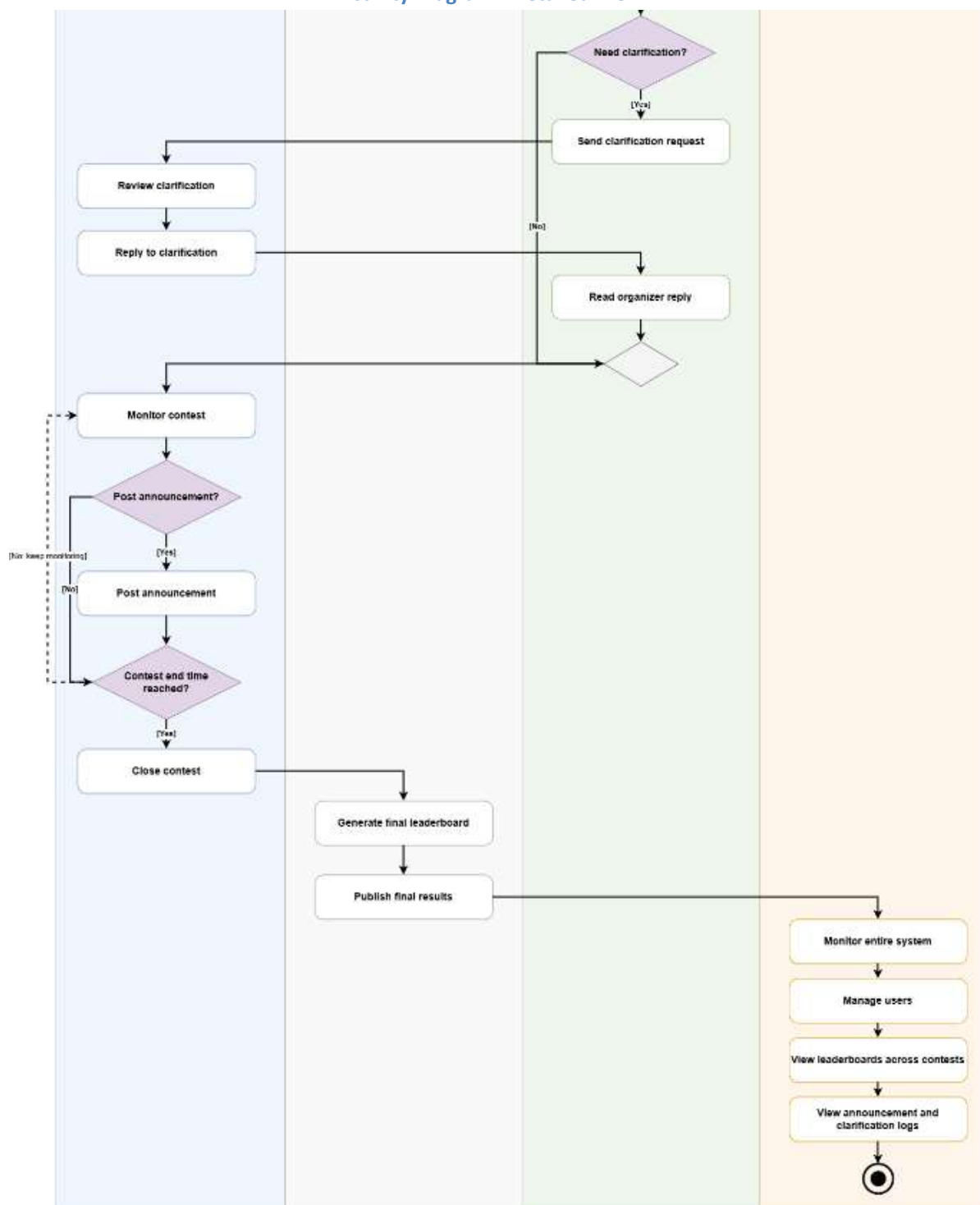
Activity Diagram - Detailed View 2



Activity Diagram - Detailed View 3

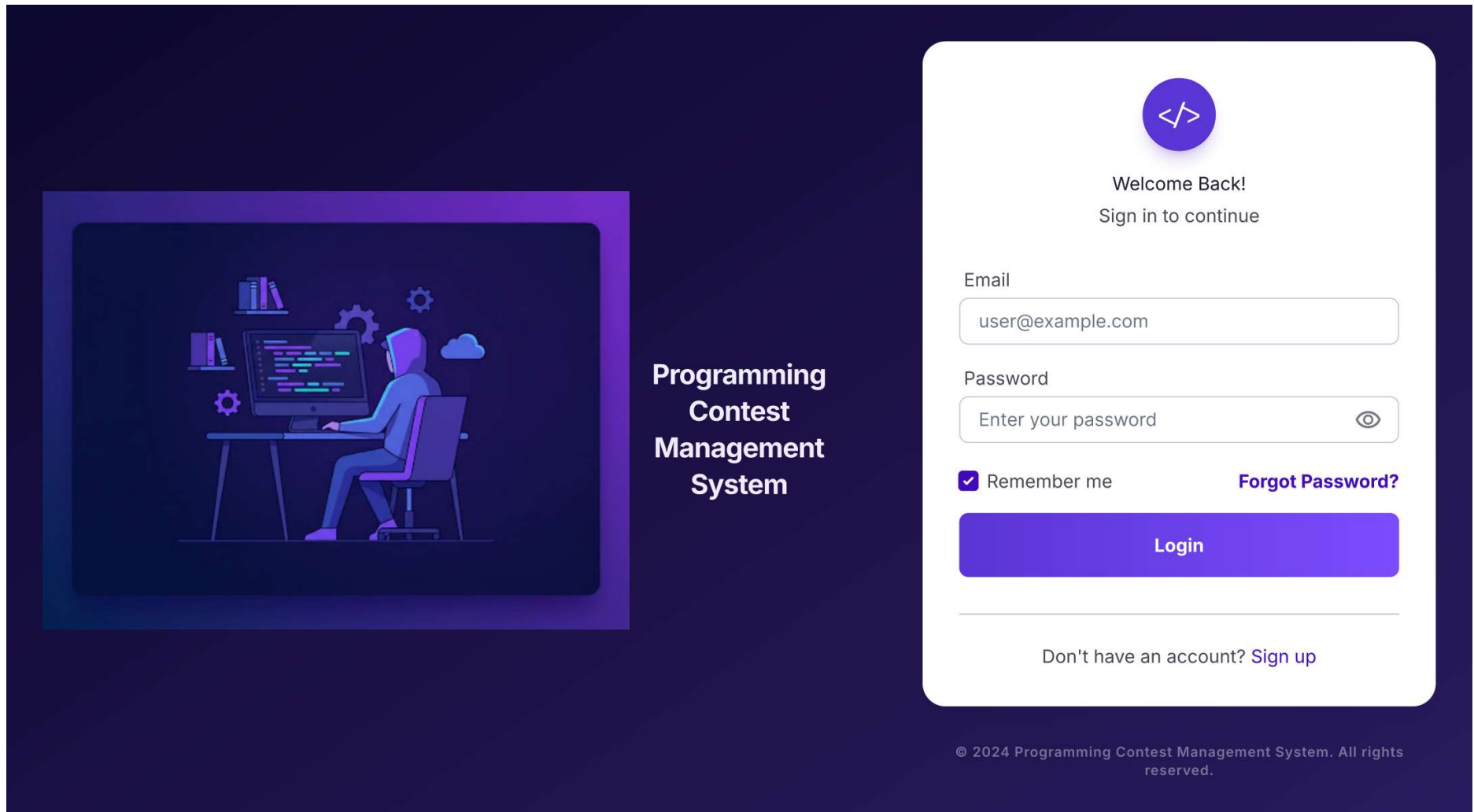


Activity Diagram - Detailed View 4



6. User Interface Planning

The following Figma screens illustrate the planned user interface for login, dashboard, contest details, problem statement, solution submission, leaderboard, and clarification activities.



UI Screen 1: Login Page

PCMS

Dashboard

Contests

Problems

Submissions

Clarifications

Leaderboard

Announcements

SYSTEM

Profile

Settings

Logout

Dashboard

Welcome back, User! 🐼

Participated Contests12

Problems Solved45

Current Rank#128

User Rating1567

Recent Submissions

View All Submissions →

Problem	Language	Verdict	Time	Memory	Date
Two Sum	C++	ACCEPTED	32 ms	1.2 MB	2 min ago
Array Rotation	Java	WRONG ANSWER	--	--	15 min ago
Longest Subarray	Python	TLE	--	--	1 hour ago
Binary Search	C++	ACCEPTED	15 ms	0.9 MB	2 hours ago
Graph BFS	C++	ACCEPTED	45 ms	1.4 MB	3 hours ago

Upcoming Contests

CodeSprint 2025

UPCOMING

25 May 2025, 10:00 AM

AlgoArena Round 8

UPCOMING

01 Jun 2025, 02:00 PM

DP Challenge

UPCOMING

10 Jun 2025, 11:00 AM

View All Contests →

Announcements

CodeSprint 2025 Schedule Released

20 May 2025

System Maintenance on 18 May

17 May 2025

New Feature: Clarification Module

15 May 2025

View All Announcements →

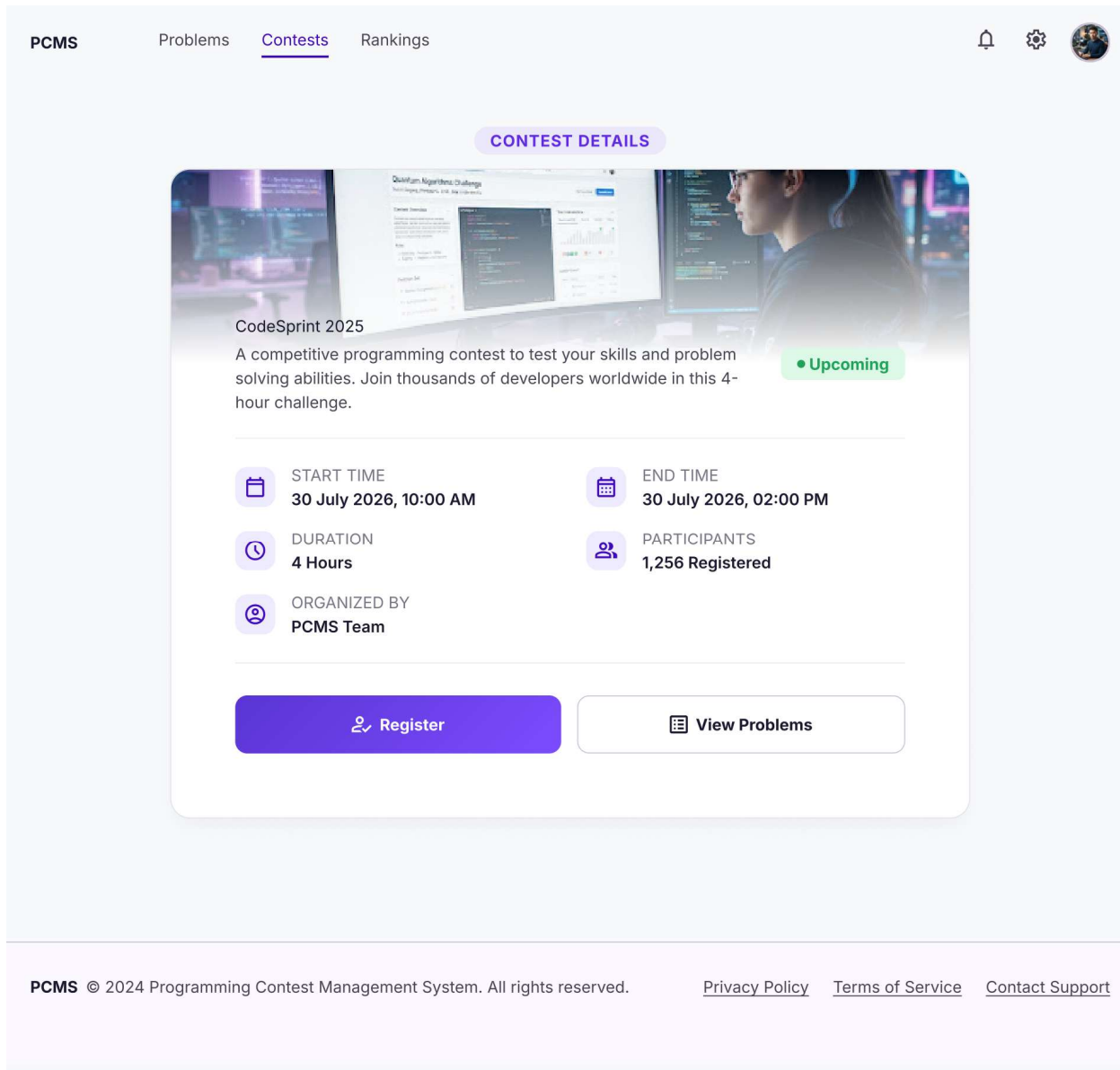
© 2024 Programming Contest Management System. All rights reserved.

Privacy Policy

Terms of Service

Contact Support

UI Screen 2: Participant Dashboard



UI Screen 3: Contest Details Page

CodeElite

Problems

Contests

Rankings



PROBLEM STATEMENT

[← Problems](#) / **A. Two Sum**

A. Two Sum

Easy

Statement

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Input

The first line contains an integer `n`.
The second line contains `n` integers.
The third line contains an integer `target`.

Output

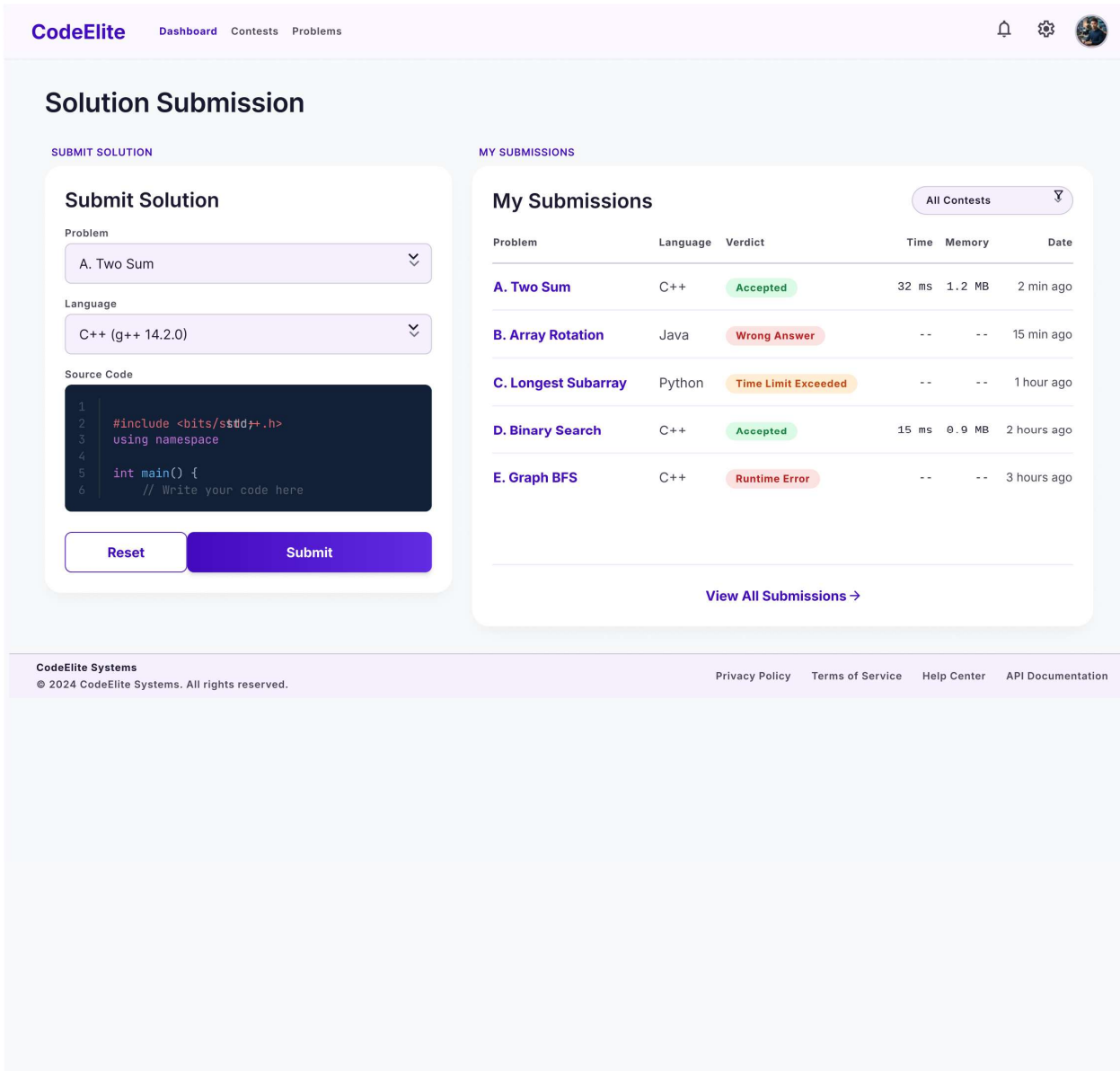
Print the two indices (0-indexed) whose sum equals target.

Sample Input

```
5
2 7 11 15 3
9
```

 Submit Solution

 Open Editor



PCMS

[Problems](#)[Contests](#)[Rankings](#)



LEADERBOARD

CodeSprint 2025

Competitive Programming Annual Event

Overall

RANK	USER	SOLVED	PENALTY	RATING
1	 Alice Johnson Lead Developer @ TechCorp	10	120	1890
2	 Bob Smith Senior Engineer @ DataSystems	9	300	1820
3	 Carol White Algorithm Specialist	9	450	1760
...				
128	 User That's you!	5	1250	1567

Showing top rankings and your position.

UPCOMING ROUNDS

Round #124

Weekly Challenge

In 2h 45m

Tomorrow

RATING TRENDS



Your performance is up by 12% this week.

QUICK RULES

Each solved problem adds 1 point.

Penalty = Finish Time + 20m per wrong submission.

PCMS

© 2024 Programming Contest Management System. All rights reserved.

[Privacy Policy](#)[Terms of Service](#)[Contact Support](#)

UI Screen 6: Leaderboard Page

PCMS

Problems

Contests

Rankings

Ask Questions

Ask Clarification

Ask a Question

CONTEST

CodeSprint 2025

QUESTION

0/1000

Type your question here...

Cancel

Submit Question

Clarifications should be related to task statements. For general technical issues, please contact site support via the help desk. Responses will appear in the "Clarification Inbox".

CONTEST TIMELINE

Remaining Time

02:45:12

PREVIOUS CLARIFICATIONS

Clarification regarding Input Format

Responded 15m ago

Problem B Memory Limit

No response yet

PCMS | © 2024 Programming Contest Management System.

[Privacy Policy](#)

[Terms of Service](#)

[Contact Support](#)

UI Screen 7: Clarification Page

7. Scenario Description

In a Programming Contest Management System, a user may organize many contests. One contest is organized by exactly one user. A user is identified by a UserID. The system also stores user name, email, password, role, institution, rating, profile image, and created date. A contest is identified by ContestID. Contest title, description, start time, end time, duration, visibility, and status are also stored. One contest may contain many problems. One problem belongs to exactly one contest. A problem is identified by ProblemID. Problem title, statement, difficulty, time limit, and memory limit are also stored. A problem may have many test cases. One test case belongs to exactly one problem. A test case is identified by TestCaseID. Test case input, expected output, and hidden status are also stored. A user may submit many submissions. One submission is submitted by exactly one user. A problem may receive many submissions. One submission is for exactly one problem. A submission is identified by SubmissionID. The system stores language, source code, verdict, execution time, memory used and submitted date. A contest may have many leaderboard records. One leaderboard record belongs to exactly one contest. A user may appear in many leaderboard records. One leaderboard record belongs to exactly one user. A leaderboard record is identified by RankID. The system stores solved problem count, penalty, and rank. A contest posts many announcements. One announcement belongs to exactly one contest. An announcement is identified by AnnouncementID. Announcement title, message, and created date are also stored. A contest may have many clarifications. One clarification belongs to exactly one contest. A user may ask many clarification questions. One clarification is asked by exactly one user. A clarification is identified by ClarificationID. The system stores question, answer, and status.

8. ER Diagram

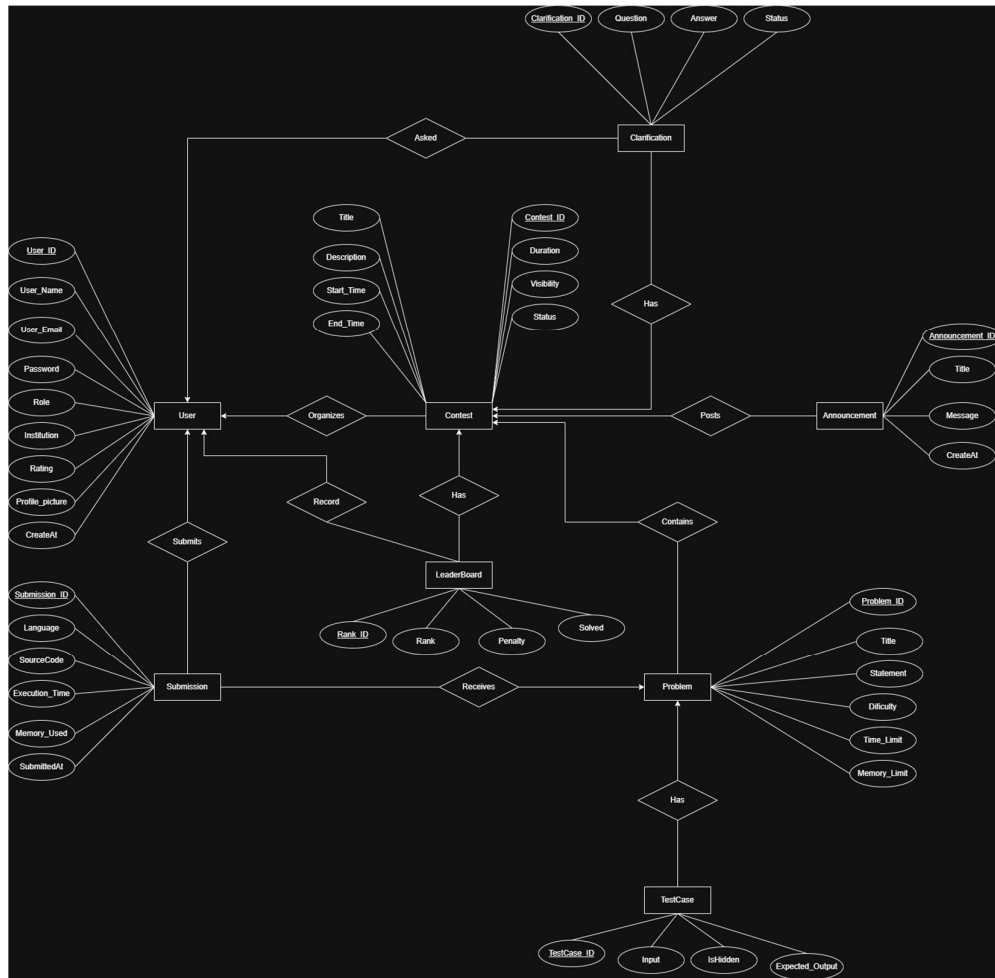


Figure 4: Entity Relationship Diagram

9. Normalization up to 3NF

Relationship 1: ORGANIZES

UNF

organizes (UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate, ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status)

1NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate, ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status

2NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status

3NF — no transitive dependency; already in 3NF:

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status

Table creation

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, **OrganizerUserID**

Relationship 2: CONTAINS

UNF

contains (ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit)

1NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit

2NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit

3NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit

Table creation

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit, **ContestID**

Relationship 3: HAS_TEST_CASE

UNF

has_test_case (ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit, TestCaseID, TestInput, ExpectedOutput, HiddenStatus)

1NF —

1. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit, TestCaseID, TestInput, ExpectedOutput, HiddenStatus

2NF —

1. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit
2. TestCaseID, TestInput, ExpectedOutput, HiddenStatus

3NF —

1. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit

2. TestCaseID, TestInput, ExpectedOutput, HiddenStatus

Table creation

1. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit
2. TestCaseID, TestInput, ExpectedOutput, HiddenStatus, **ProblemID**

Relationship 4: SUBMITS

UNF

submits(UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate, SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate)

1NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate, SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate

2NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate

3NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate

Table creation

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate, **UserID**

Relationship 5: RECEIVES_SUBMISSION

UNF

receives_submission(ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit, SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate)

1NF —

1. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit, SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate

2NF —

1. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit
2. SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate

3NF —

1. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit
2. SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate

Table creation

1. ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit
2. SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate, **ProblemID**

Relationship 6: HAS_LEADERBOARD

UNF

has_leaderboard(ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, RankID, SolvedProblemCount, Penalty, RankPosition)

1NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, RankID, SolvedProblemCount, Penalty, RankPosition

2NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. RankID, SolvedProblemCount, Penalty, RankPosition

3NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. RankID, SolvedProblemCount, Penalty, RankPosition

Table creation

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. RankID, SolvedProblemCount, Penalty, RankPosition, **ContestID**

Relationship 7: APPEARS_IN_LEADERBOARD

UNF

appears_in_leaderboard (UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate, RankID, SolvedProblemCount, Penalty, RankPosition)

1NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate, RankID, SolvedProblemCount, Penalty, RankPosition

2NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. RankID, SolvedProblemCount, Penalty, RankPosition

3NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. RankID, SolvedProblemCount, Penalty, RankPosition

Table creation

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. RankID, SolvedProblemCount, Penalty, RankPosition, **UserID**

Relationship 8: POSTS

UNF

posts (ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, AnnouncementID, AnnouncementTitle, Message, CreatedDate)

1NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, AnnouncementID, AnnouncementTitle, Message, CreatedDate

2NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. AnnouncementID, AnnouncementTitle, Message, CreatedDate

3NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. AnnouncementID, AnnouncementTitle, Message, CreatedDate

Table creation

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. AnnouncementID, AnnouncementTitle, Message, CreatedDate, **ContestID**

Relationship 9: HAS_CLARIFICATION

UNF

has_clarification (ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, ClarificationID, Question, Answer, ClarificationStatus)

1NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, ClarificationID, Question, Answer, ClarificationStatus

2NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. ClarificationID, Question, Answer, ClarificationStatus

3NF —

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. ClarificationID, Question, Answer, ClarificationStatus

Table creation

1. ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status
2. ClarificationID, Question, Answer, Status, **ContestID**

Relationship 10: ASKS

UNF

asks (UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate, ClarificationID, Question, Answer, Status)

1NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate, ClarificationID, Question, Answer, Status

2NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. ClarificationID, Question, Answer, Status

3NF —

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. ClarificationID, Question, Answer, Status

Table creation

1. UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate
2. ClarificationID, Question, Answer, Status, **UserID**

Final Table:

1. CONTEST_USER (UserID, UserName, Email, Password, Role, Institution, Rating, ProfileImage, CreatedDate)
2. CONTEST (ContestID, ContestTitle, Description, StartTime, EndTime, Duration, Visibility, Status, **OrganizerUserID**)
3. PROBLEM (ProblemID, ProblemTitle, ProblemStatement, Difficulty, TimeLimit, MemoryLimit, **ContestID**)
4. TEST_CASE (TestCaseID, TestInput, ExpectedOutput, HiddenStatus, **ProblemID**)
5. SUBMISSION (SubmissionID, Language, SourceCode, Verdict, ExecutionTime, MemoryUsed, SubmittedDate, **UserID**, **ProblemID**)
6. LEADERBOARD (RankID, SolvedProblemCount, Penalty, RankPosition, **ContestID**, **UserID**)
7. ANNOUNCEMENT (AnnouncementID, AnnouncementTitle, Message, CreatedDate, **ContestID**)
8. CLARIFICATION (ClarificationID, Question, Answer, Status, **ContestID**, **UserID**)

10. Schema Diagram

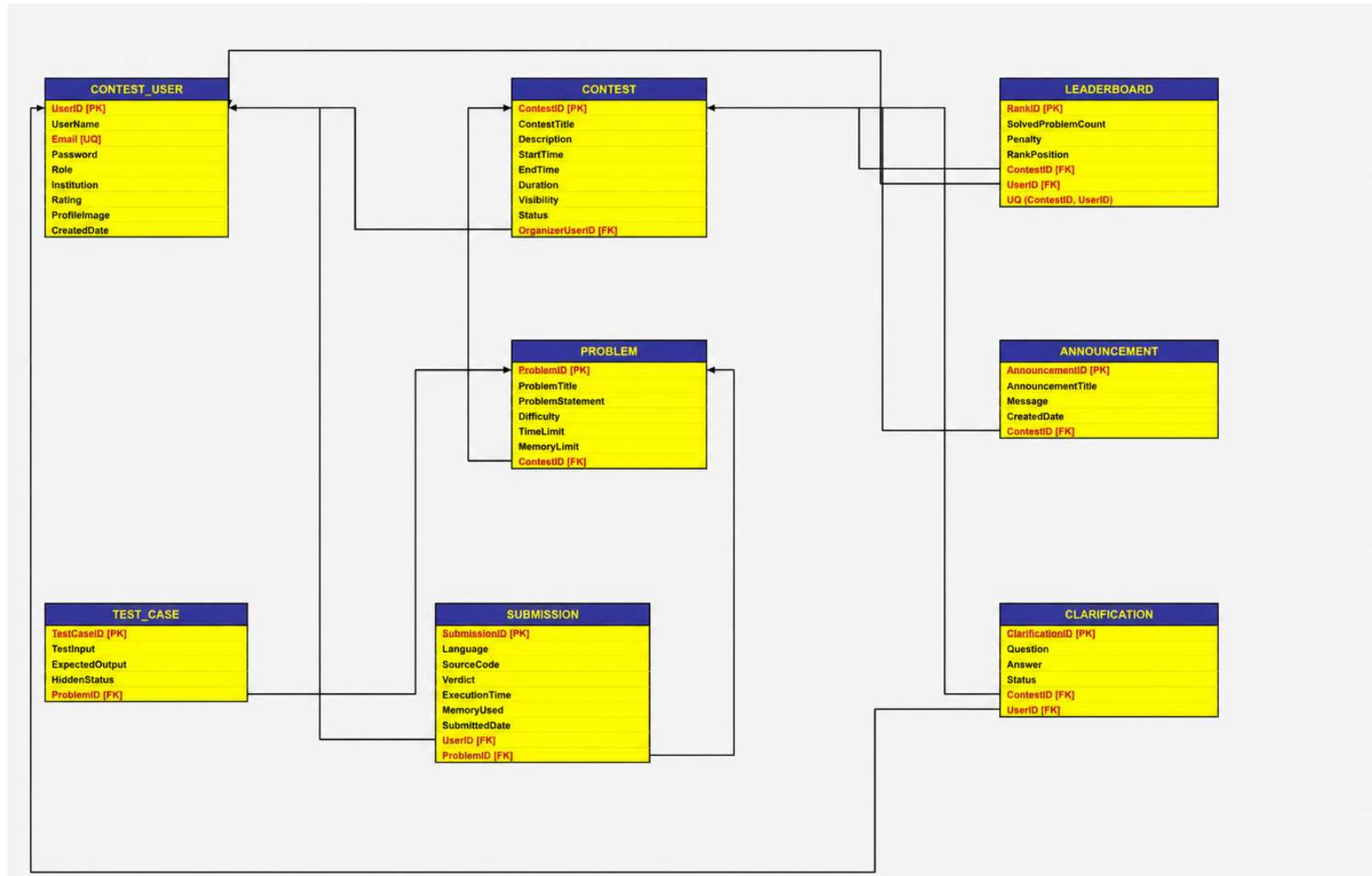


Figure 5: Schema Diagram after Normalization

11. Table Creation Using SQL

TABLE CONTEST_USER:

```
CREATE TABLE CONTEST_USER (  
  UserID NUMBER PRIMARY KEY,  
  UserName VARCHAR2(100) NOT NULL,  
  Email VARCHAR2(150) NOT NULL UNIQUE,  
  Password VARCHAR2(255) NOT NULL,  
  UserRole VARCHAR2(20) NOT NULL,  
  Institution VARCHAR2(150),  
  Rating NUMBER(8,2),  
  ProfileImage VARCHAR2(255),  
  CreatedDate DATE  
);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the command `DESC CONTEST_USER;`. Below the command, the table structure for `CONTEST_USER` is displayed. The table has the following columns:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CONTEST_USER	USERID	Number	-	-	-	1	-	-	-
	USERNAME	Varchar2	100	-	-	-	-	-	-
	EMAIL	Varchar2	150	-	-	-	-	-	-
	PASSWORD	Varchar2	255	-	-	-	-	-	-
	USERROLE	Varchar2	20	-	-	-	-	-	-
	INSTITUTION	Varchar2	150	-	-	-	✓	-	-
	RATING	Number	-	8	2	-	✓	-	-
	PROFILEIMAGE	Varchar2	255	-	-	-	✓	-	-
	CREATEDDATE	Date	7	-	-	-	✓	-	-

The interface also shows the 'Autocommit' checkbox checked, the 'Display' dropdown set to '10', and the 'Save' and 'Run' buttons. The bottom status bar indicates the language is 'en-us' and the application is 'Express 2.1.0.0.39'.

TABLE CONTEST:

```
CREATE TABLE CONTEST (  
  ContestID NUMBER PRIMARY KEY,  
  ContestTitle VARCHAR2(150) NOT NULL,  
  Description VARCHAR2(1000),  
  StartTime DATE NOT NULL,  
  EndTime DATE NOT NULL,  
  Duration NUMBER,  
  Visibility VARCHAR2(20),  
  Status VARCHAR2(20),  
  OrganizerUserID NUMBER,  
  CONSTRAINT fk_contest_user FOREIGN KEY(OrganizerUserID)  
  REFERENCES CONTEST_USER(UserID)  
);
```

SQL Commands

127.0.0.1:8081/apex/?p=4500:1003:236880212216664::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

DESC CONTEST;

Results Explain Describe Saved SQL History

Object Type TABLE Object CONTEST

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CONTEST	CONTESTID	Number	-	-	-	1	-	-	-
	CONTESTTITLE	Varchar2	150	-	-	-	-	-	-
	DESCRIPTION	Varchar2	1000	-	-	-	✓	-	-
	STARTTIME	Date	7	-	-	-	-	-	-
	ENDTIME	Date	7	-	-	-	-	-	-
	DURATION	Number	-	-	-	-	✓	-	-
	VISIBILITY	Varchar2	20	-	-	-	✓	-	-
	STATUS	Varchar2	20	-	-	-	✓	-	-
	ORGANIZERUSERID	Number	-	-	-	-	✓	-	-

1 - 9

Language: en-us

Application Express 2.1.0.0.39
Copyright © 1999, 2006, Oracle. All rights reserved.

TABLE PROBLEM:

```
CREATE TABLE PROBLEM (
ProblemID NUMBER PRIMARY KEY,
ProblemTitle VARCHAR2(150) NOT NULL,
ProblemStatement CLOB NOT NULL,
Difficulty VARCHAR2(20),
TimeLimit NUMBER,
MemoryLimit NUMBER,
ContestID NUMBER,
CONSTRAINT fk_problem_contest FOREIGN KEY(ContestID)
REFERENCES CONTEST(ContestID)
);
```

SQL Commands

127.0.0.1:8081/apex/f?p=4500:1003:236880212216664::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

DESC PROBLEM;

Results Explain Describe Saved SQL History

Object Type TABLE Object PROBLEM

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PROBLEM	PROBLEMID	Number	-	-	-	1	-	-	-
	PROBLEMTITLE	Varchar2	150	-	-	-	-	-	-
	PROBLEMSTATEMENT	Clob	4000	-	-	-	-	-	-
	DIFFICULTY	Varchar2	20	-	-	-	✓	-	-
	TIMELIMIT	Number	-	-	-	-	✓	-	-
	MEMORYLIMIT	Number	-	-	-	-	✓	-	-
	CONTESTID	Number	-	-	-	-	✓	-	-
1 - 7									

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Language: en-us

12:45 AM 7/29/2026

TABLE TEST_CASE:

```
CREATE TABLE TEST_CASE (
  TestCaseID NUMBER PRIMARY KEY,
  TestInput CLOB NOT NULL,
  ExpectedOutput CLOB NOT NULL,
  HiddenStatus CHAR(1),
  ProblemID NUMBER,
  CONSTRAINT fk_testcase_problem FOREIGN KEY(ProblemID)
  REFERENCES PROBLEM(ProblemID)
);
```

SQL Commands

127.0.0.1:8081/apex/f?p=4500:1003:236880212216664::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

DESC TEST_CASE;

Results Explain Describe Saved SQL History

Object Type TABLE Object TEST_CASE

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
TEST_CASE	TESTCASEID	Number	-	-	-	1	-	-	-
	TESTINPUT	Clob	4000	-	-	-	-	-	-
	EXPECTEDOUTPUT	Clob	4000	-	-	-	-	-	-
	HIDDENSTATUS	Char	1	-	-	-	✓	-	-
	PROBLEMID	Number	-	-	-	-	✓	-	-
1 - 5									

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Language: en-us

12:46 AM 7/29/2026

TABLE SUBMISSION:

```
CREATE TABLE SUBMISSION (  
SubmissionID NUMBER PRIMARY KEY,  
Language VARCHAR2(40) NOT NULL,  
SourceCode CLOB NOT NULL,  
Verdict VARCHAR2(40),  
ExecutionTime NUMBER(10,3),  
MemoryUsed NUMBER,  
SubmittedDate DATE,  
UserID NUMBER,  
ProblemID NUMBER,  
CONSTRAINT fk_submission_user FOREIGN KEY(UserID)  
REFERENCES CONTEST_USER(UserID),  
CONSTRAINT fk_submission_problem FOREIGN KEY(ProblemID)  
REFERENCES PROBLEM(ProblemID)  
);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window is active, displaying the command 'DESC SUBMISSION;'. Below the command, the table structure for 'SUBMISSION' is shown. The table has 10 columns: SUBMISSIONID (Number, Primary Key), LANGUAGE (Varchar2), SOURCECODE (Clob), VERDICT (Varchar2), EXECUTIONTIME (Number), MEMORYUSED (Number), SUBMITTEDDATE (Date), USERID (Number), PROBLEMID (Number), and a row for constraints. The constraints are: PRIMARY KEY on SUBMISSIONID, FOREIGN KEY on USERID referencing CONTEST_USER, and FOREIGN KEY on PROBLEMID referencing PROBLEM.

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SUBMISSION	SUBMISSIONID	Number	-	-	-	1	-	-	-
	LANGUAGE	Varchar2	40	-	-	-	-	-	-
	SOURCECODE	Clob	4000	-	-	-	-	-	-
	VERDICT	Varchar2	40	-	-	-	✓	-	-
	EXECUTIONTIME	Number	-	10	3	-	✓	-	-
	MEMORYUSED	Number	-	-	-	-	✓	-	-
	SUBMITTEDDATE	Date	7	-	-	-	✓	-	-
	USERID	Number	-	-	-	-	✓	-	-
	PROBLEMID	Number	-	-	-	-	✓	-	-
1 - 9									

TABLE LEADERBOARD:

```
CREATE TABLE LEADERBOARD (  
RankID NUMBER PRIMARY KEY,  
SolvedProblemCount NUMBER,  
Penalty NUMBER,  
RankPosition NUMBER,  
ContestID NUMBER,  
UserID NUMBER,  
CONSTRAINT uq_leaderboard_user UNIQUE(ContestID,UserID),  
CONSTRAINT fk_leaderboard_contest FOREIGN KEY(ContestID)  
REFERENCES CONTEST(ContestID),  
CONSTRAINT fk_leaderboard_user FOREIGN KEY(UserID)  
REFERENCES CONTEST_USER(UserID)  
);
```

SQL Commands

127.0.0.1:8081/apex/?p=4500:1003:236880212216664::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

DESC LEADERBOARD;

Results Explain Describe Saved SQL History

Object Type TABLE Object LEADERBOARD

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
LEADERBOARD	RANKID	Number	-	-	-	1	-	-	-
	SOLVEDPROBLEMCOUNT	Number	-	-	-	-	✓	-	-
	PENALTY	Number	-	-	-	-	✓	-	-
	RANKPOSITION	Number	-	-	-	-	✓	-	-
	CONTESTID	Number	-	-	-	-	✓	-	-
	USERID	Number	-	-	-	-	✓	-	-

1 - 6

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Language: en-us

12:48 AM 7/29/2026

TABLE ANNOUNCEMENT:

CREATE TABLE ANNOUNCEMENT (
 AnnouncementID NUMBER PRIMARY KEY,
 AnnouncementTitle VARCHAR2(150) NOT NULL,
 Message VARCHAR2(2000) NOT NULL,
 CreatedDate DATE,
 ContestID NUMBER,
 CONSTRAINT fk_announcement_contest FOREIGN KEY(ContestID)
 REFERENCES CONTEST(ContestID)
);

SQL Commands

127.0.0.1:8081/apex/?p=4500:1003:236880212216664::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

DESC ANNOUNCEMENT;

Results Explain Describe Saved SQL History

Object Type TABLE Object ANNOUNCEMENT

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ANNOUNCEMENT	ANNOUNCEMENTID	Number	-	-	-	1	-	-	-
	ANNOUNCEMENTTITLE	Varchar2	150	-	-	-	-	-	-
	MESSAGE	Varchar2	2000	-	-	-	-	-	-
	CREATEDDATE	Date	7	-	-	-	✓	-	-
	CONTESTID	Number	-	-	-	-	✓	-	-

1 - 5

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Language: en-us

12:49 AM 7/29/2026

TABLE CLARIFICATION:

```
CREATE TABLE CLARIFICATION (  
ClarificationID NUMBER PRIMARY KEY,  
Question VARCHAR2(2000) NOT NULL,  
Answer VARCHAR2(2000),  
Status VARCHAR2(20),  
ContestID NUMBER,  
UserID NUMBER,  
CONSTRAINT fk_clarification_contest FOREIGN KEY(ContestID)  
REFERENCES CONTEST(ContestID),  
CONSTRAINT fk_clarification_user FOREIGN KEY(UserID)  
REFERENCES CONTEST_USER(UserID)  
);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the command: `DESC CLARIFICATION;`. Below the command window, the 'Describe' tab is active, displaying the table structure for the **CLARIFICATION** table. The table has six columns: **CLARIFICATIONID** (Number, Primary Key), **QUESTION** (Varchar2, 2000), **ANSWER** (Varchar2, 2000), **STATUS** (Varchar2, 20), **CONTESTID** (Number), and **USERID** (Number). The **CLARIFICATIONID** column is marked as the Primary Key. The **ANSWER**, **STATUS**, **CONTESTID**, and **USERID** columns are marked as nullable with checkmarks. The interface also shows the 'Save' and 'Run' buttons, and the 'Results' tab is visible at the bottom.

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CLARIFICATION	CLARIFICATIONID	Number	-	-	-	1	-	-	-
	QUESTION	Varchar2	2000	-	-	-	-	-	-
	ANSWER	Varchar2	2000	-	-	-	✓	-	-
	STATUS	Varchar2	20	-	-	-	✓	-	-
	CONTESTID	Number	-	-	-	-	✓	-	-
	USERID	Number	-	-	-	-	✓	-	-

1 - 6

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

12. Data Insertion Using SQL

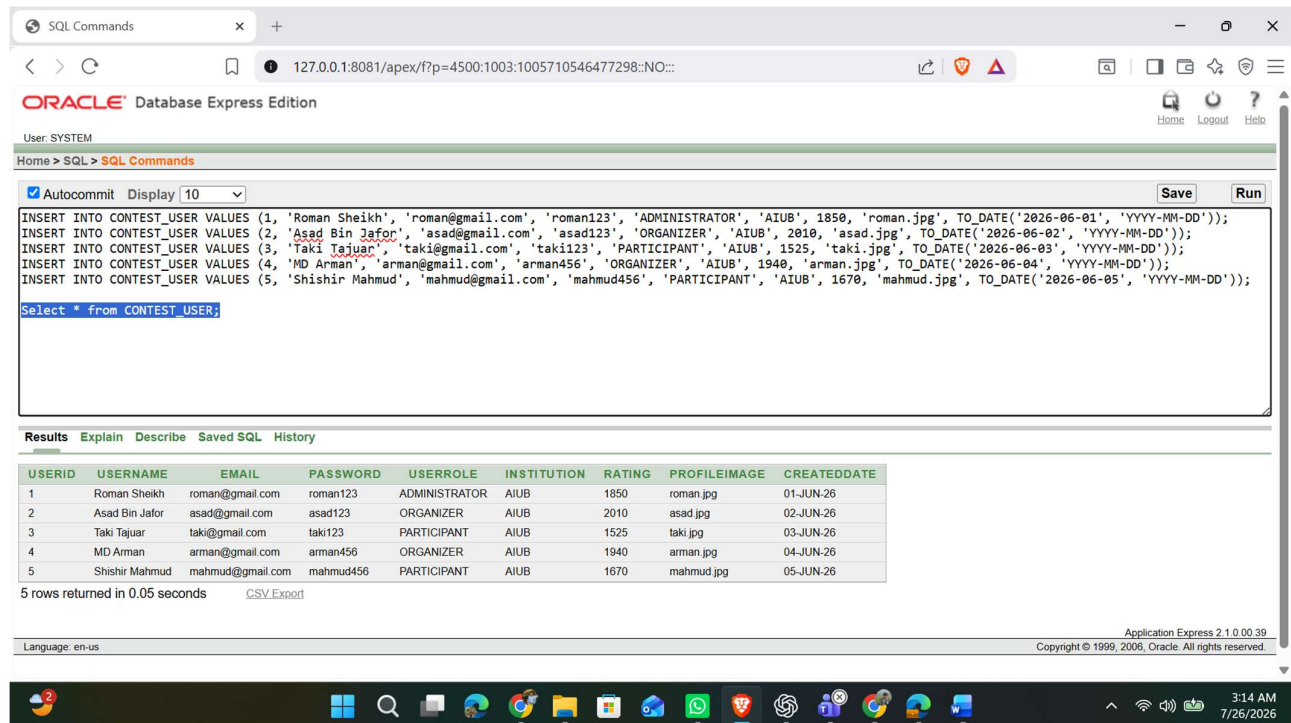
```
INSERT INTO CONTEST_USER VALUES (1, 'Roman Sheikh', 'roman@gmail.com', 'roman123',  
'ADMINISTRATOR', 'AIUB', 1850, 'roman.jpg', TO_DATE('2026-06-01', 'YYYY-MM-DD'));
```

```
INSERT INTO CONTEST_USER VALUES (2, 'Asad Bin Jafor', 'asad@gmail.com', 'asad123', 'ORGANIZER',  
'AIUB', 2010, 'asad.jpg', TO_DATE('2026-06-02', 'YYYY-MM-DD'));
```

```
INSERT INTO CONTEST_USER VALUES (3, 'Taki Tajuar', 'taki@gmail.com', 'taki123', 'PARTICIPANT',  
'AIUB', 1525, 'taki.jpg', TO_DATE('2026-06-03', 'YYYY-MM-DD'));
```

```
INSERT INTO CONTEST_USER VALUES (4, 'MD Arman', 'arman@gmail.com', 'arman456',  
'ORGANIZER', 'AIUB', 1940, 'arman.jpg', TO_DATE('2026-06-04', 'YYYY-MM-DD'));
```

```
INSERT INTO CONTEST_USER VALUES (5, 'Shishir Mahmud', 'mahmud@gmail.com', 'mahmud456',  
'PARTICIPANT', 'AIUB', 1670, 'mahmud.jpg', TO_DATE('2026-06-05', 'YYYY-MM-DD'));
```



The screenshot shows the Oracle Database Express Edition SQL Commands window. The SQL Commands tab is active, displaying five INSERT statements and a SELECT statement. The statements are executed successfully, and the results are shown in a table below. The table has 9 columns: USERID, USERNAME, EMAIL, PASSWORD, USERROLE, INSTITUTION, RATING, PROFILEIMAGE, and CREATEDDATE. The results show 5 rows of user data.

USERID	USERNAME	EMAIL	PASSWORD	USERROLE	INSTITUTION	RATING	PROFILEIMAGE	CREATEDDATE
1	Roman Sheikh	roman@gmail.com	roman123	ADMINISTRATOR	AIUB	1850	roman.jpg	01-JUN-26
2	Asad Bin Jafor	asad@gmail.com	asad123	ORGANIZER	AIUB	2010	asad.jpg	02-JUN-26
3	Taki Tajuar	taki@gmail.com	taki123	PARTICIPANT	AIUB	1525	taki.jpg	03-JUN-26
4	MD Arman	arman@gmail.com	arman456	ORGANIZER	AIUB	1940	arman.jpg	04-JUN-26
5	Shishir Mahmud	mahmud@gmail.com	mahmud456	PARTICIPANT	AIUB	1670	mahmud.jpg	05-JUN-26

5 rows returned in 0.05 seconds

```
INSERT INTO CONTEST VALUES (101, 'AIUB Programming Contest 2026', 'University programming  
contest', TO_DATE('2026-07-20 10:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-07-20 13:00', 'YYYY-  
MM-DD HH24:MI'), 180, 'PUBLIC', 'COMPLETED', 2);
```

```
INSERT INTO CONTEST VALUES (102, 'BRAC Programming Contest 2026', 'Online algorithm contest',  
TO_DATE('2026-07-22 14:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-07-22 17:00', 'YYYY-MM-DD  
HH24:MI'), 180, 'PUBLIC', 'ONGOING', 2);
```

```
INSERT INTO CONTEST VALUES (103, 'NSU Programming Contest 2026', 'Programming contest in  
technology', TO_DATE('2026-07-30 10:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-07-30 14:00',  
'YYYY-MM-DD HH24:MI'), 240, 'PUBLIC', 'UPCOMING', 4);
```

```
INSERT INTO CONTEST VALUES (104, 'Dhaka Inter-University Coding Challenge', 'Coding contest for  
university students', TO_DATE('2026-08-05 09:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-08-05  
14:00', 'YYYY-MM-DD HH24:MI'), 300, 'PRIVATE', 'UPCOMING', 4);
```

```
INSERT INTO CONTEST VALUES (105, 'Independence Day Online Contest', 'National online programming  
contest', TO_DATE('2026-03-26 20:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-03-26 23:00', 'YYYY-  
MM-DD HH24:MI'), 180, 'PUBLIC', 'COMPLETED', 2);
```

SQL Commands

127.0.0.1:8081/apex/f?p=4500:1003:1005710546477298::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```

INSERT INTO CONTEST VALUES (101, 'AIUB Programming Contest 2026', 'University programming contest', TO_DATE('2026-07-20 10:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-07-20 13:00', 'YYYY-MM-DD HH24:MI'), 180, 'PUBLIC', 'COMPLETED', 2);
INSERT INTO CONTEST VALUES (102, 'BRAC Programming Contest 2026', 'Online algorithm contest', TO_DATE('2026-07-22 14:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-07-22 17:00', 'YYYY-MM-DD HH24:MI'), 180, 'PUBLIC', 'ONGOING', 2);
INSERT INTO CONTEST VALUES (103, 'NSU Programming Contest 2026', 'Programming contest in technology', TO_DATE('2026-07-30 10:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-07-30 14:00', 'YYYY-MM-DD HH24:MI'), 240, 'PUBLIC', 'UPCOMING', 4);
INSERT INTO CONTEST VALUES (104, 'Dhaka Inter-University Coding Challenge', 'Coding contest for university students', TO_DATE('2026-08-05 09:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-08-05 14:00', 'YYYY-MM-DD HH24:MI'), 300, 'PRIVATE', 'UPCOMING', 4);
INSERT INTO CONTEST VALUES (105, 'Independence Day Online Contest', 'National online programming contest', TO_DATE('2026-03-26 20:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-03-26 23:00', 'YYYY-MM-DD HH24:MI'), 180, 'PUBLIC', 'COMPLETED', 2);

Select * from CONTEST;

```

Results Explain Describe Saved SQL History

CONTESTID	CONTESTTITLE	DESCRIPTION	STARTTIME	ENDTIME	DURATION	VISIBILITY	STATUS	ORGANIZERUSERID
101	AIUB Programming Contest 2026	University programming contest	20-JUL-26	20-JUL-26	180	PUBLIC	COMPLETED	2
102	BRAC Programming Contest 2026	Online algorithm contest	22-JUL-26	22-JUL-26	180	PUBLIC	ONGOING	2
103	NSU Programming Contest 2026	Programming contest in technology	30-JUL-26	30-JUL-26	240	PUBLIC	UPCOMING	4
104	Dhaka Inter-University Coding Challenge	Coding contest for university students	05-AUG-26	05-AUG-26	300	PRIVATE	UPCOMING	4
105	Independence Day Online Contest	National online programming contest	26-MAR-26	26-MAR-26	180	PUBLIC	COMPLETED	2

5 rows returned in 0.00 seconds CSV Export

Language: en-us Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.

INSERT INTO PROBLEM VALUES (201, 'Sum of Two Numbers', 'Read two integers and print their sum.', 'EASY', 1000, 64, 101);

INSERT INTO PROBLEM VALUES (202, 'Dynamic Sequence Queries', 'Process update and range sum queries efficiently.', 'HARD', 3000, 256, 102);

INSERT INTO PROBLEM VALUES (203, 'Balanced Brackets', 'Determine whether the bracket sequence is balanced.', 'EASY', 1000, 64, 103);

INSERT INTO PROBLEM VALUES (204, 'Shortest Path Delivery', 'Find the minimum delivery cost between two cities.', 'MEDIUM', 2000, 128, 104);

INSERT INTO PROBLEM VALUES (205, 'River Crossing Optimization', 'Find the minimum time required to cross the river.', 'HARD', 2500, 128, 105);

SQL Commands

127.0.0.1:8081/apex/f?p=4500:1003:1005710546477298::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```

INSERT INTO PROBLEM VALUES (201, 'Sum of Two Numbers', 'Read two integers and print their sum.', 'EASY', 1000, 64, 101);
INSERT INTO PROBLEM VALUES (202, 'Dynamic Sequence Queries', 'Process update and range sum queries efficiently.', 'HARD', 3000, 256, 102);
INSERT INTO PROBLEM VALUES (203, 'Balanced Brackets', 'Determine whether the bracket sequence is balanced.', 'EASY', 1000, 64, 103);
INSERT INTO PROBLEM VALUES (204, 'Shortest Path Delivery', 'Find the minimum delivery cost between two cities.', 'MEDIUM', 2000, 128, 104);
INSERT INTO PROBLEM VALUES (205, 'River Crossing Optimization', 'Find the minimum time required to cross the river.', 'HARD', 2500, 128, 105);

Select * from PROBLEM;

```

Results Explain Describe Saved SQL History

PROBLEMID	PROBLEMTITLE	PROBLEMSTATEMENT	DIFFICULTY	TIMELIMIT	MEMORYLIMIT	CONTESTID
201	Sum of Two Numbers	Read two integers and print their sum.	EASY	1000	64	101
202	Dynamic Sequence Queries	Process update and range sum queries efficiently.	HARD	3000	256	102
203	Balanced Brackets	Determine whether the bracket sequence is balanced.	EASY	1000	64	103
204	Shortest Path Delivery	Find the minimum delivery cost between two cities.	MEDIUM	2000	128	104
205	River Crossing Optimization	Find the minimum time required to cross the river.	HARD	2500	128	105

5 rows returned in 0.01 seconds CSV Export

Language: en-us Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.

```

INSERT INTO TEST_CASE VALUES (301, '12 30', '42', 'N', 201);
INSERT INTO TEST_CASE VALUES (302, '5 3 1 2 3 4 5', '9 22', 'Y', 202);
INSERT INTO TEST_CASE VALUES (303, '[({})]', 'YES', 'N', 203);
INSERT INTO TEST_CASE VALUES (304, '4 5 1 2 4 2 3 2 3 4 1', '7', 'Y', 204);
INSERT INTO TEST_CASE VALUES (305, '4 1 2 5 10', '17', 'Y', 205);

```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains five INSERT statements for the TEST_CASE table. Below the commands, the 'Results' tab is active, displaying a table with 5 rows of data. The table has columns: TESTCASEID, TESTINPUT, EXPECTEDOUTPUT, HIDDENSTATUS, and PROBLEMID. The data is as follows:

TESTCASEID	TESTINPUT	EXPECTEDOUTPUT	HIDDENSTATUS	PROBLEMID
301	12 30	42	N	201
302	5 3 1 2 3 4 5	9 22	Y	202
303	[({})]	YES	N	203
304	4 5 1 2 4 2 3 2 3 4 1	7	Y	204
305	4 1 2 5 10	17	Y	205

5 rows returned in 0.01 seconds

```

INSERT INTO SUBMISSION VALUES (401, 'C++', 'Program to calculate the sum', 'ACCEPTED', 12.500,
1024, TO_DATE('2026-07-20 10:35', 'YYYY-MM-DD HH24:MI'), 3, 201);
INSERT INTO SUBMISSION VALUES (402, 'Java', 'Program to process sequence queries',
'TIME_LIMIT_EXCEEDED', 3000.00, 32768, TO_DATE('2026-07-22 14:40', 'YYYY-MM-DD HH24:MI'), 3,
202);
INSERT INTO SUBMISSION VALUES (403, 'Python', 'Program to check balanced brackets', 'ACCEPTED',
48.750, 8192, TO_DATE('2026-07-30 10:25', 'YYYY-MM-DD HH24:MI'), 5, 203);
INSERT INTO SUBMISSION VALUES (404, 'C++', 'Program to find the shortest path',
'WRONG_ANSWER', 35.000, 2048, TO_DATE('2026-08-05 10:10', 'YYYY-MM-DD HH24:MI'), 5, 204);
INSERT INTO SUBMISSION VALUES (405, 'C', 'Program to solve river crossing', 'PENDING', 0.000, 0,
TO_DATE('2026-03-26 21:05', 'YYYY-MM-DD HH24:MI'), 3, 205);

```

Oracle Database Express Edition SQL Commands window. The SQL command entered is:

```

INSERT INTO SUBMISSION VALUES (401, 'C++', 'Program to calculate the sum', 'ACCEPTED', 12.500, 1024, TO_DATE('2026-07-20 10:35', 'YYYY-MM-DD HH24:MI'), 3, 201);
INSERT INTO SUBMISSION VALUES (402, 'Java', 'Program to process sequence queries', 'TIME_LIMIT_EXCEEDED', 3000.00, 32768, TO_DATE('2026-07-22 14:40', 'YYYY-MM-DD HH24:MI'), 3, 202);
INSERT INTO SUBMISSION VALUES (403, 'Python', 'Program to check balanced brackets', 'ACCEPTED', 48.750, 8192, TO_DATE('2026-07-30 10:25', 'YYYY-MM-DD HH24:MI'), 5, 203);
INSERT INTO SUBMISSION VALUES (404, 'C++', 'Program to find the shortest path', 'WRONG_ANSWER', 35.000, 2048, TO_DATE('2026-08-05 10:10', 'YYYY-MM-DD HH24:MI'), 5, 204);
INSERT INTO SUBMISSION VALUES (405, 'C', 'Program to solve river crossing', 'PENDING', 0.000, 0, TO_DATE('2026-03-26 21:05', 'YYYY-MM-DD HH24:MI'), 3, 205);

Select * from SUBMISSION;

```

The results table shows 5 rows returned in 0.01 seconds:

SUBMISSIONID	LANGUAGE	SOURCECODE	VERDICT	EXECUTIONTIME	MEMORYUSED	SUBMITTEDDATE	USERID	PROBLEMID
401	C++	Program to calculate the sum	ACCEPTED	12.5	1024	20-JUL-26	3	201
402	Java	Program to process sequence queries	TIME_LIMIT_EXCEEDED	3000	32768	22-JUL-26	3	202
403	Python	Program to check balanced brackets	ACCEPTED	48.75	8192	30-JUL-26	5	203
404	C++	Program to find the shortest path	WRONG_ANSWER	35	2048	05-AUG-26	5	204
405	C	Program to solve river crossing	PENDING	0	0	26-MAR-26	3	205

INSERT INTO LEADERBOARD VALUES (501, 1, 35, 1, 101, 3);

INSERT INTO LEADERBOARD VALUES (502, 0, 0, 2, 102, 3);

INSERT INTO LEADERBOARD VALUES (503, 1, 25, 1, 103, 5);

INSERT INTO LEADERBOARD VALUES (504, 0, 0, 2, 104, 5);

INSERT INTO LEADERBOARD VALUES (505, 1, 65, 1, 105, 3);

Oracle Database Express Edition SQL Commands window. The SQL command entered is:

```

INSERT INTO LEADERBOARD VALUES (501, 1, 35, 1, 101, 3);
INSERT INTO LEADERBOARD VALUES (502, 0, 0, 2, 102, 3);
INSERT INTO LEADERBOARD VALUES (503, 1, 25, 1, 103, 5);
INSERT INTO LEADERBOARD VALUES (504, 0, 0, 2, 104, 5);
INSERT INTO LEADERBOARD VALUES (505, 1, 65, 1, 105, 3);

Select * from LEADERBOARD;

```

The results table shows 5 rows returned in 0.01 seconds:

RANKID	SOLVEDPROBLEMCOUNT	PENALTY	RANKPOSITION	CONTESTID	USERID
501	1	35	1	101	3
502	0	0	2	102	3
503	1	25	1	103	5
504	0	0	2	104	5
505	1	65	1	105	3

INSERT INTO ANNOUNCEMENT VALUES (601, 'Contest Registration Opened', 'Registration is now open for all participants.', TO_DATE('2026-07-01', 'YYYY-MM-DD'), 101);

INSERT INTO ANNOUNCEMENT VALUES (602, 'Rule Update', 'Penalty will be calculated for accepted problems.', TO_DATE('2026-07-22', 'YYYY-MM-DD'), 102);

INSERT INTO ANNOUNCEMENT VALUES (603, 'Contest Starting Soon', 'Participants should join thirty minutes early.', TO_DATE('2026-07-30', 'YYYY-MM-DD'), 103);

INSERT INTO ANNOUNCEMENT VALUES (604, 'Team List Published', 'The final list of participating teams has been published.', TO_DATE('2026-08-01', 'YYYY-MM-DD'), 104);

INSERT INTO ANNOUNCEMENT VALUES (605, 'Final Result Published', 'The final contest result has been published.', TO_DATE('2026-03-27', 'YYYY-MM-DD'), 105);

The screenshot shows the Oracle Database Express Edition SQL Commands window. The SQL Commands tab is active, displaying a series of INSERT statements for the ANNOUNCEMENT table. The statements are as follows:

```

INSERT INTO ANNOUNCEMENT VALUES (601, 'Contest Registration Opened', 'Registration is now open for all participants.', TO_DATE('2026-07-01', 'YYYY-MM-DD'), 101);
INSERT INTO ANNOUNCEMENT VALUES (602, 'Rule Update', 'Penalty will be calculated for accepted problems.', TO_DATE('2026-07-22', 'YYYY-MM-DD'), 102);
INSERT INTO ANNOUNCEMENT VALUES (603, 'Contest Starting Soon', 'Participants should join thirty minutes early.', TO_DATE('2026-07-30', 'YYYY-MM-DD'), 103);
INSERT INTO ANNOUNCEMENT VALUES (604, 'Team List Published', 'The final list of participating teams has been published.', TO_DATE('2026-08-01', 'YYYY-MM-DD'), 104);
INSERT INTO ANNOUNCEMENT VALUES (605, 'Final Result Published', 'The final contest result has been published.', TO_DATE('2026-03-27', 'YYYY-MM-DD'), 105);

```

Below the SQL Commands, the Results tab is active, showing a table with 5 rows returned in 0.00 seconds. The table has the following columns: ANNOUNCEMENTID, ANNOUNCEMENTTITLE, MESSAGE, CREATEDDATE, and CONTESTID.

ANNOUNCEMENTID	ANNOUNCEMENTTITLE	MESSAGE	CREATEDDATE	CONTESTID
601	Contest Registration Opened	Registration is now open for all participants.	01-JUL-26	101
602	Rule Update	Penalty will be calculated for accepted problems.	22-JUL-26	102
603	Contest Starting Soon	Participants should join thirty minutes early.	30-JUL-26	103
604	Team List Published	The final list of participating teams has been published.	01-AUG-26	104
605	Final Result Published	The final contest result has been published.	27-MAR-26	105

At the bottom of the window, the status bar shows the language as en-us, the application version as 2.1.0.00.39, and the copyright as 1999, 2006, Oracle. All rights reserved. The system clock shows 3:27 AM on 7/26/2026.

INSERT INTO CLARIFICATION VALUES (701, 'Can 64-bit integers be used?', 'Yes, 64-bit integers can be used.', 'ANSWERED', 101, 3);

INSERT INTO CLARIFICATION VALUES (702, 'Are query ranges one-indexed?', NULL, 'OPEN', 102, 3);

INSERT INTO CLARIFICATION VALUES (703, 'Does an empty string count as balanced?', 'An empty string will not occur.', 'CLOSED', 103, 5);

INSERT INTO CLARIFICATION VALUES (704, 'Are parallel edges allowed in the graph?', 'Yes, parallel edges are allowed.', 'ANSWERED', 104, 5);

INSERT INTO CLARIFICATION VALUES (705, 'Can two people cross together?', 'At most two people can cross together.', 'ANSWERED', 105, 3);

COMMIT;

SQL Commands

127.0.0.1:8081/apex/?p=4500:1003:1005710546477298::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit

Display 10

Save

Run

```
INSERT INTO CLARIFICATION VALUES (701, 'Can 64-bit integers be used?', 'Yes, 64-bit integers can be used.', 'ANSWERED', 101, 3);
INSERT INTO CLARIFICATION VALUES (702, 'Are query ranges one-indexed?', NULL, 'OPEN', 102, 3);
INSERT INTO CLARIFICATION VALUES (703, 'Does an empty string count as balanced?', 'An empty string will not occur.', 'CLOSED', 103, 5);
INSERT INTO CLARIFICATION VALUES (704, 'Are parallel edges allowed in the graph?', 'Yes, parallel edges are allowed.', 'ANSWERED', 104, 5);
INSERT INTO CLARIFICATION VALUES (705, 'Can two people cross together?', 'At most two people can cross together.', 'ANSWERED', 105, 3);

Select * from CLARIFICATION;
```

Results

Explain

Describe

Saved SQL

History

CLARIFICATIONID	QUESTION	ANSWER	STATUS	CONTESTID	USERID
701	Can 64-bit integers be used?	Yes, 64-bit integers can be used.	ANSWERED	101	3
702	Are query ranges one-indexed?	-	OPEN	102	3
703	Does an empty string count as balanced?	An empty string will not occur.	CLOSED	103	5
704	Are parallel edges allowed in the graph?	Yes, parallel edges are allowed.	ANSWERED	104	5
705	Can two people cross together?	At most two people can cross together.	ANSWERED	105	3

5 rows returned in 0.00 seconds

CSV Export

Application Express 2.1.0.00.39

Copyright © 1999, 2006, Oracle. All rights reserved.

Language: en-us

3:29 AM 7/26/2026

13. Query Writing Using PL/SQL

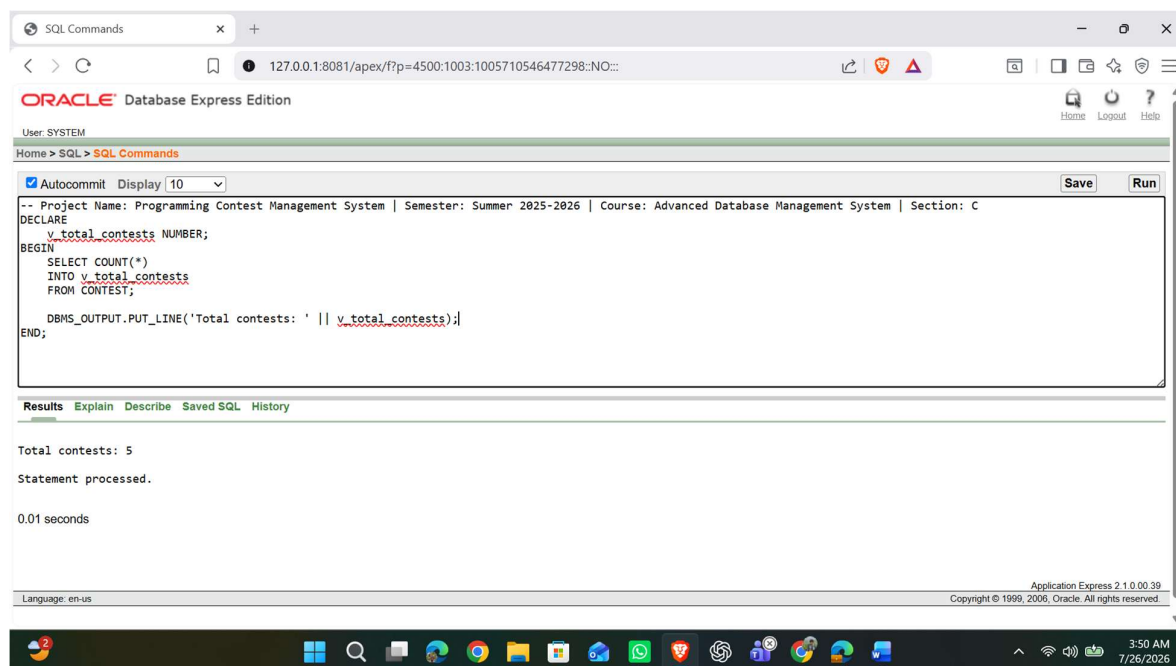
13.1 Basic PL/SQL

Category: Variables

Q1: Write a PL/SQL program that stores the total number of contests in a variable and displays it.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C

```
DECLARE
    v_total_contests NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_total_contests
    FROM CONTEST;
    DBMS_OUTPUT.PUT_LINE('Total contests: ' || v_total_contests);
END;
```



Q2: Write a PL/SQL program that stores a participant name and rating in two variables and displays both values.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C

```
DECLARE
    v_participant_name CONTEST_USER.UserName%TYPE;
    v_participant_rating CONTEST_USER.Rating%TYPE;
```

```

BEGIN

    SELECT UserName, Rating
    INTO v_participant_name, v_participant_rating
    FROM CONTEST_USER
    WHERE Email = 'taki@gmail.com'
       AND UserRole = 'PARTICIPANT';

    DBMS_OUTPUT.PUT_LINE('Participant: ' || v_participant_name);
    DBMS_OUTPUT.PUT_LINE('Rating: ' || NVL(v_participant_rating, 0));

EXCEPTION

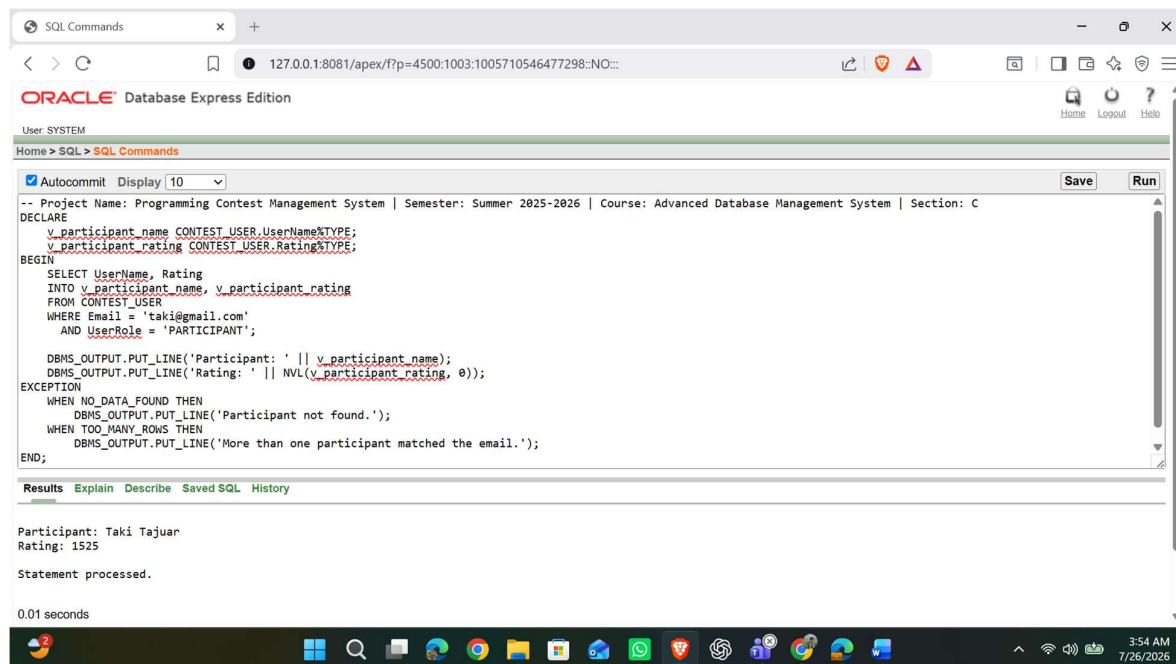
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Participant not found.');
```

```

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one participant matched the email.');
```

```

END;
```



Category: Operators

Q3: Write a PL/SQL program that calculates a participant rating after adding a 75-point bonus using an arithmetic operator.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

DECLARE

    v_current_rating CONTEST_USER.Rating%TYPE;

    v_bonus NUMBER := 75;

    v_calculated_rating NUMBER;

BEGIN
```

```

SELECT NVL(Rating, 0)
INTO v_current_rating
FROM CONTEST_USER
WHERE Email = 'mahmud@gmail.com'
AND UserRole = 'PARTICIPANT';

v_calculated_rating := v_current_rating + v_bonus;

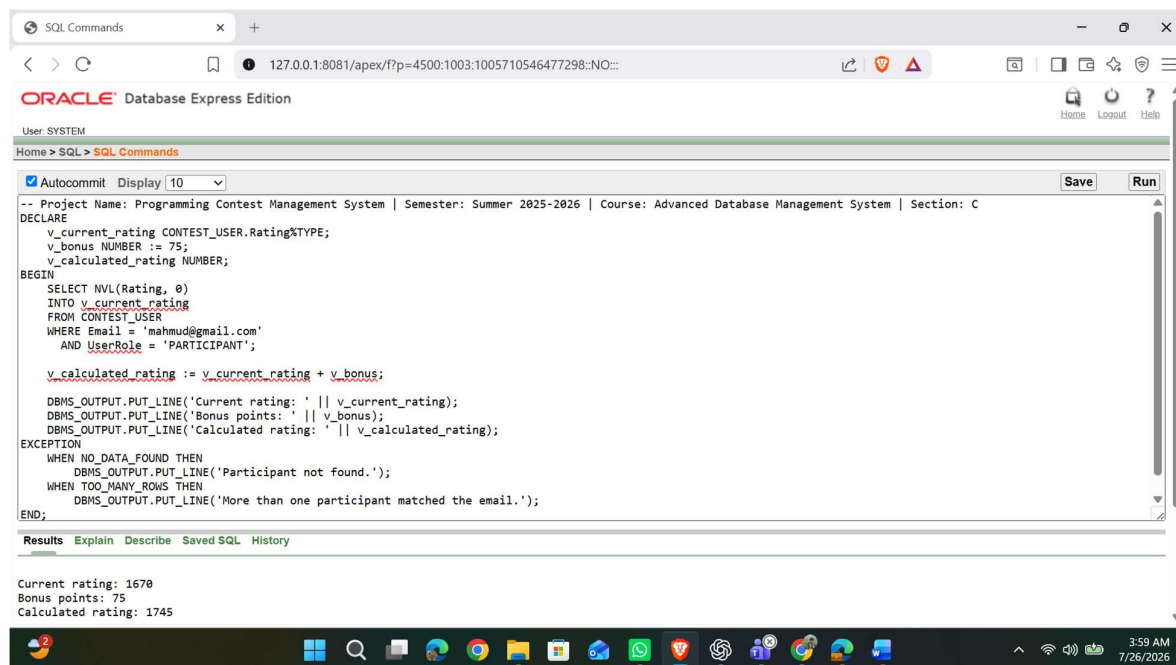
DBMS_OUTPUT.PUT_LINE('Current rating: ' || v_current_rating);
DBMS_OUTPUT.PUT_LINE('Bonus points: ' || v_bonus);
DBMS_OUTPUT.PUT_LINE('Calculated rating: ' || v_calculated_rating);
EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Participant not found.');
```

```

WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('More than one participant matched the email.');
```

```

END;
```



Q4: Write a PL/SQL program that uses comparison and logical operators to check whether a contest lasts at least 180 minutes and is upcoming or ongoing.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

DECLARE

    v_duration CONTEST.Duration%TYPE;
```

```

v_status CONTEST.Status%TYPE;

BEGIN

SELECT Duration, Status
INTO v_duration, v_status
FROM CONTEST
WHERE ContestTitle = 'AIUB Programming Contest 2026';

IF v_duration >= 180
AND (v_status = 'COMPLETED' OR v_status = 'ONGOING') THEN
    DBMS_OUTPUT.PUT_LINE('Contest meets the duration and status conditions.');
```

ELSE

```

    DBMS_OUTPUT.PUT_LINE('Contest does not meet the duration and status conditions.');
```

END IF;

EXCEPTION

```

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Contest not found.');
```

WHEN TOO_MANY_ROWS THEN

```

        DBMS_OUTPUT.PUT_LINE('More than one contest matched the title.');
```

END;

SQL Commands

127.0.0.1:8081/apex/f?p=4500:1003:1005710546477298::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
DECLARE
v_duration CONTEST.Duration%TYPE;
v_status CONTEST.Status%TYPE;
BEGIN
SELECT Duration, Status
INTO v_duration, v_status
FROM CONTEST
WHERE ContestTitle = 'AIUB Programming Contest 2026';

IF v_duration >= 180
AND (v_status = 'COMPLETED' OR v_status = 'ONGOING') THEN
    DBMS_OUTPUT.PUT_LINE('Contest meets the duration and status conditions.');
```

ELSE

```

    DBMS_OUTPUT.PUT_LINE('Contest does not meet the duration and status conditions.');
```

END IF;

EXCEPTION

```

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Contest not found.');
```

WHEN TOO_MANY_ROWS THEN

```

        DBMS_OUTPUT.PUT_LINE('More than one contest matched the title.');
```

END;

Category: Single-Row Functions

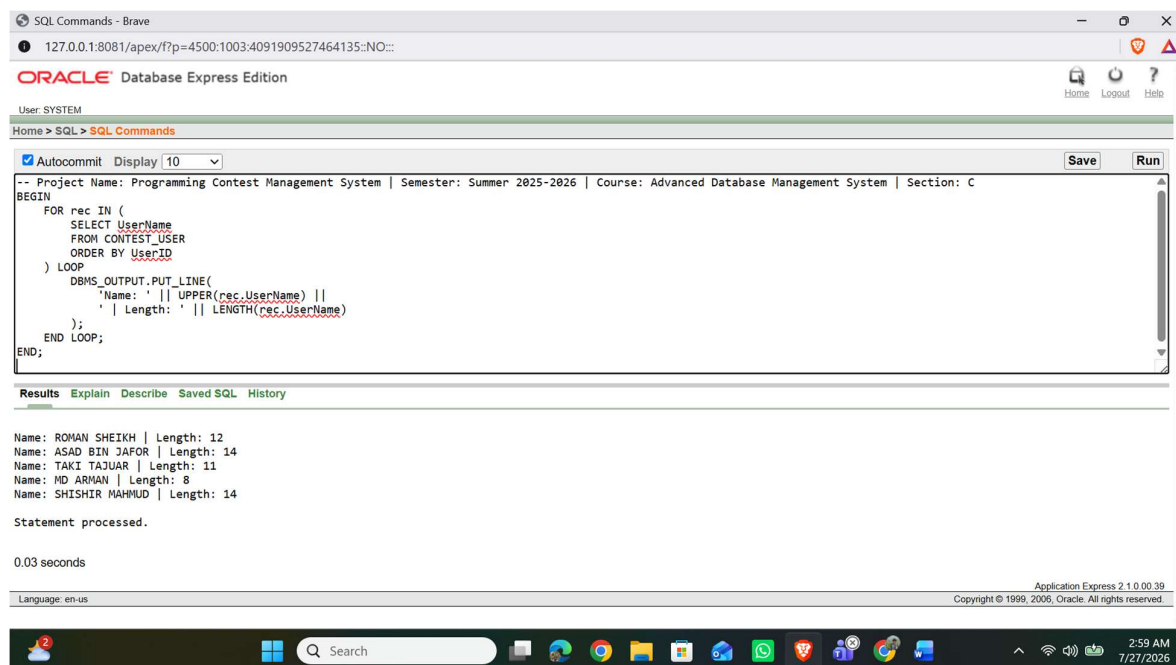
Q5: Write a PL/SQL program that displays every user name in uppercase together with the length of the name.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

BEGIN
  FOR rec IN (
    SELECT UserName
    FROM CONTEST_USER
    ORDER BY UserID
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Name: ' || UPPER(rec.UserName) ||
      ' | Length: ' || LENGTH(rec.UserName)
    );
  END LOOP;
END;

```



Q6: Write a PL/SQL program that displays each contest title with its start date formatted as DD-MON-YYYY HH24:MI.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

BEGIN
  FOR rec IN (
    SELECT ContestTitle, StartTime
    FROM CONTEST
    ORDER BY StartTime
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Contest: ' || rec.ContestTitle ||

```

```

        ' | Starts: ' || TO_CHAR(rec.StartTime, 'DD-MON-YYYY HH24:MI')
    );
END LOOP;
END;

```

SQL Commands - Brave

127.0.0.1:8081/apex/?p=4500:1003:4091909527464135::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
BEGIN
  FOR rec IN (
    SELECT ContestTitle, StartTime
    FROM CONTEST
    ORDER BY StartTime
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Contest: ' || rec.ContestTitle ||
      ' | Starts: ' || TO_CHAR(rec.StartTime, 'DD-MON-YYYY HH24:MI')
    );
  END LOOP;
END;

```

Results Explain Describe Saved SQL History

```

Contest: Independence Day Online Contest | Starts: 26-MAR-2026 20:00
Contest: AIUB Programming Contest 2026 | Starts: 20-JUL-2026 10:00
Contest: BRAC Programming Contest 2026 | Starts: 22-JUL-2026 14:00
Contest: NSU Programming Contest 2026 | Starts: 30-JUL-2026 10:00
Contest: Dhaka Inter-University Coding Challenge | Starts: 05-AUG-2026 09:00

```

Statement processed.

0.00 seconds

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Category: Group Functions

Q7: Write a PL/SQL program that counts the number of submissions for each problem.

```

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C
BEGIN
  FOR rec IN (
    SELECT p.ProblemTitle,
           COUNT(s.SubmissionID) AS SubmissionCount
    FROM PROBLEM p
    LEFT JOIN SUBMISSION s
      ON p.ProblemID = s.ProblemID
    GROUP BY p.ProblemID, p.ProblemTitle
    ORDER BY p.ProblemID
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Problem: ' || rec.ProblemTitle ||
      ' | Submissions: ' || rec.SubmissionCount
    );
  END LOOP;
END;

```

SQL Commands - Brave

127.0.0.1:8081/apex/?p=4500:1003:4091909527464135::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
BEGIN
  FOR rec IN (
    SELECT p.ProblemTitle,
           COUNT(s.SubmissionID) AS SubmissionCount
    FROM PROBLEM p
    LEFT JOIN SUBMISSION s
    ON p.ProblemID = s.ProblemID
    GROUP BY p.ProblemID, p.ProblemTitle
    ORDER BY p.ProblemID
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Problem: ' || rec.ProblemTitle ||
      ' Submissions: ' || rec.SubmissionCount
    );
  END LOOP;
END;
```

Results Explain Describe Saved SQL History

Problem: Sum of Two Numbers | Submissions: 1
 Problem: Dynamic Sequence Queries | Submissions: 1
 Problem: Balanced Brackets | Submissions: 1
 Problem: Shortest Path Delivery | Submissions: 1
 Problem: River Crossing Optimization | Submissions: 1

Statement processed.

0.01 seconds

Q8: Write a PL/SQL program that calculates and displays the average rating of all participant users.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

DECLARE

v_average_rating NUMBER;

BEGIN

SELECT AVG(NVL(Rating, 0))

INTO v_average_rating

FROM CONTEST_USER

WHERE UserRole = 'PARTICIPANT';

DBMS_OUTPUT.PUT_LINE('Average participant rating: ' || v_average_rating);

END;

SQL Commands - Brave

127.0.0.1:8081/apex/?p=4500:1003:4091909527464135::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
DECLARE
  v_average_rating NUMBER;
BEGIN
  SELECT AVG(NVL(Rating, 0))
  INTO v_average_rating
  FROM CONTEST_USER
  WHERE UserRole = 'PARTICIPANT';
  DBMS_OUTPUT.PUT_LINE('Average participant rating: ' || v_average_rating);
END;
```

Results Explain Describe Saved SQL History

Average participant rating: 1597.5

Statement processed.

0.01 seconds

Language: en-us

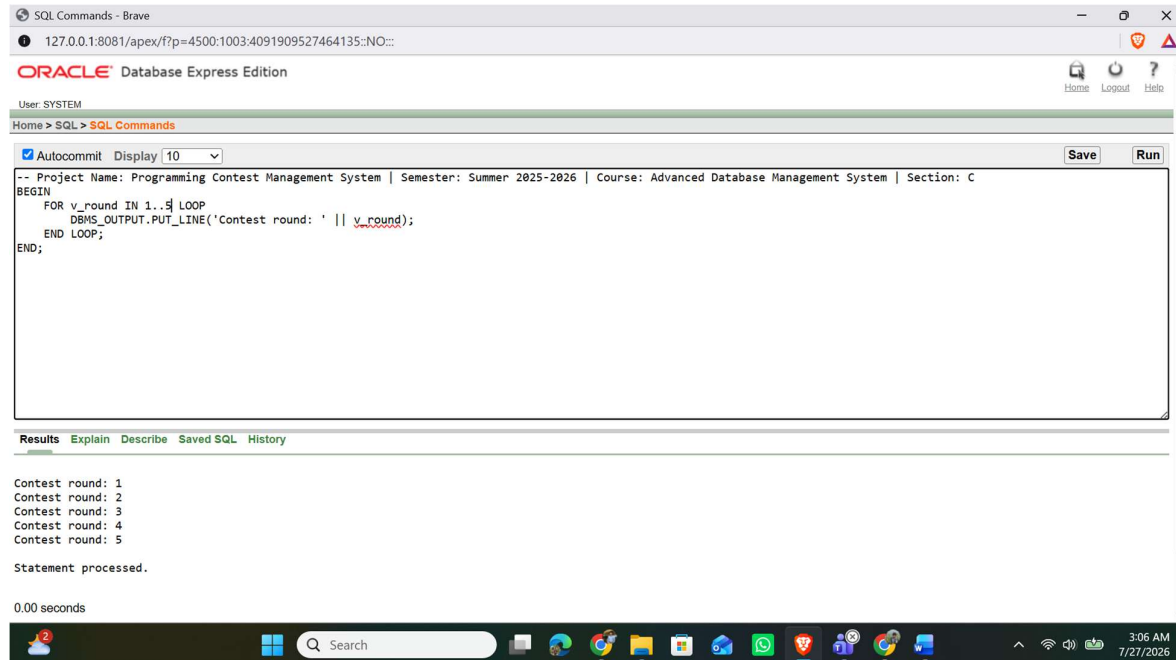
Application Express 2.1.0.00.39
 Copyright © 1999, 2006, Oracle. All rights reserved.

Category: Loops

Q9: Write a PL/SQL program that uses a numeric FOR loop to display five contest rounds.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C

```
BEGIN
    FOR v_round IN 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE('Contest round: ' || v_round);
    END LOOP;
END;
```



Q10: Write a PL/SQL program that uses a WHILE loop to display the first five user IDs in ascending order.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C

```
DECLARE
    CURSOR c_users IS
        SELECT UserID
        FROM CONTEST_USER
        ORDER BY UserID;
    v_user_id CONTEST_USER.UserID%TYPE;
    v_counter NUMBER := 0;
BEGIN
    OPEN c_users;
    FETCH c_users INTO v_user_id;

    WHILE c_users%FOUND AND v_counter < 5 LOOP
        v_counter := v_counter + 1;
        DBMS_OUTPUT.PUT_LINE(
            'User position ' || v_counter || ': ID ' || v_user_id
        );
        FETCH c_users INTO v_user_id;
    END LOOP;
```

```

CLOSE c_users;
END;

```

The screenshot shows the Oracle Database Express Edition SQL Commands window. The project name is 'Programming Contest Management System', semester is 'Summer 2025-2026', course is 'Advanced Database Management System', and section is 'C'. The user is 'SYSTEM'. The SQL Commands window shows a PL/SQL program that declares a cursor 'c_users' to select 'UserID' from 'CONTEST_USER' ordered by 'UserID'. It then opens the cursor, fetches the first row into 'v_user_id', and enters a loop that prints the user position and ID for the first 5 users. The program ends by closing the cursor. The output shows the first 5 users: 'User position 1: ID 1', 'User position 2: ID 2', 'User position 3: ID 3', 'User position 4: ID 4', and 'User position 5: ID 5'. The status bar indicates 'Statement processed'.

```

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
DECLARE
  CURSOR c_users IS
    SELECT UserID
    FROM CONTEST_USER
    ORDER BY UserID;
  v_user_id CONTEST_USER.UserID%TYPE;
  v_counter NUMBER := 0;
BEGIN
  OPEN c_users;
  FETCH c_users INTO v_user_id;

  WHILE c_users%FOUND AND v_counter < 5 LOOP
    v_counter := v_counter + 1;
    DBMS_OUTPUT.PUT_LINE(
      'User position ' || v_counter || ': ID ' || v_user_id
    );
    FETCH c_users INTO v_user_id;
  END LOOP;

  CLOSE c_users;
END;

Results Explain Describe Saved SQL History

User position 1: ID 1
User position 2: ID 2
User position 3: ID 3
User position 4: ID 4
User position 5: ID 5

Statement processed.

```

Category: Conditional Statements

Q11: Write a PL/SQL program that uses IF-ELSIF-ELSE to classify a participant rating as Beginner, Intermediate, or Advanced.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

DECLARE
  v_name CONTEST_USER.UserName%TYPE;
  v_rating CONTEST_USER.Rating%TYPE;
BEGIN
  SELECT UserName, NVL(Rating, 0)
  INTO v_name, v_rating
  FROM CONTEST_USER
  WHERE Email = 'taki@gmail.com'
  AND UserRole = 'PARTICIPANT';

  IF v_rating < 1200 THEN
    DBMS_OUTPUT.PUT_LINE(v_name || ' is a Beginner participant.');
```

```

  ELSIF v_rating < 1800 THEN
```

```

    DBMS_OUTPUT.PUT_LINE(v_name || ' is an Intermediate participant.');
```

```

  ELSE
```

```

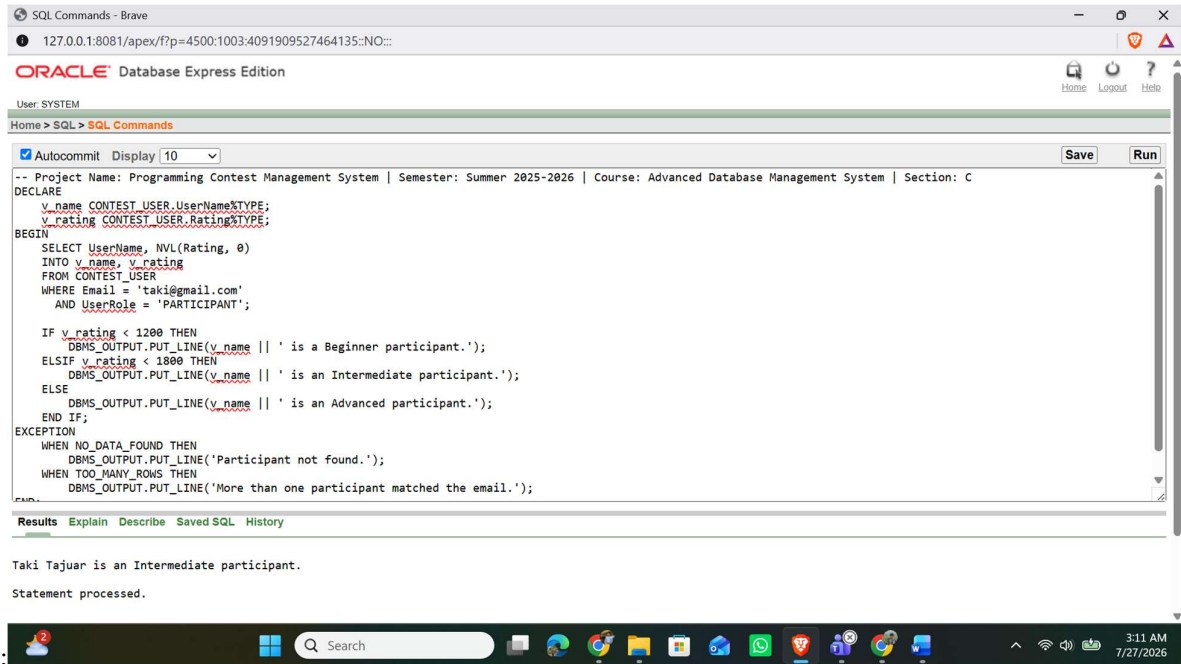
    DBMS_OUTPUT.PUT_LINE(v_name || ' is an Advanced participant.');
```

```

END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Participant not found.');
```

```

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one participant matched the email.');
```



```

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
DECLARE
    v_name CONTEST_USER.UserName%TYPE;
    v_rating CONTEST_USER.Rating%TYPE;
BEGIN
    SELECT UserName, NVL(Rating, 0)
    INTO v_name, v_rating
    FROM CONTEST_USER
    WHERE Email = 'taki@gmail.com'
    AND UserRole = 'PARTICIPANT';

    IF v_rating < 1200 THEN
        DBMS_OUTPUT.PUT_LINE(v_name || ' is a Beginner participant.');
```

```

    ELSIF v_rating < 1800 THEN
        DBMS_OUTPUT.PUT_LINE(v_name || ' is an Intermediate participant.');
```

```

    ELSE
        DBMS_OUTPUT.PUT_LINE(v_name || ' is an Advanced participant.');
```

```

    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Participant not found.');
```

```

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one participant matched the email.');
```

END;

Q12: Write a PL/SQL program that uses a CASE statement to display a message for a contest status.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

DECLARE
    v_title CONTEST.ContestTitle%TYPE;
    v_status CONTEST.Status%TYPE;
BEGIN
    SELECT ContestTitle, Status
    INTO v_title, v_status
    FROM CONTEST
    WHERE ContestTitle = 'AIUB Programming Contest 2026';

    DBMS_OUTPUT.PUT_LINE('Contest: ' || v_title);

    CASE v_status
        WHEN 'UPCOMING' THEN
            DBMS_OUTPUT.PUT_LINE('The contest has not started yet.');
```

```

        WHEN 'ONGOING' THEN
            DBMS_OUTPUT.PUT_LINE('The contest is currently running.');
```

```

        WHEN 'COMPLETED' THEN
            DBMS_OUTPUT.PUT_LINE('The contest has finished.');
```

```

        WHEN 'CANCELLED' THEN
            DBMS_OUTPUT.PUT_LINE('The contest was cancelled.');
```

```

ELSE
    DBMS_OUTPUT.PUT_LINE('The contest status is unknown.');
```

```

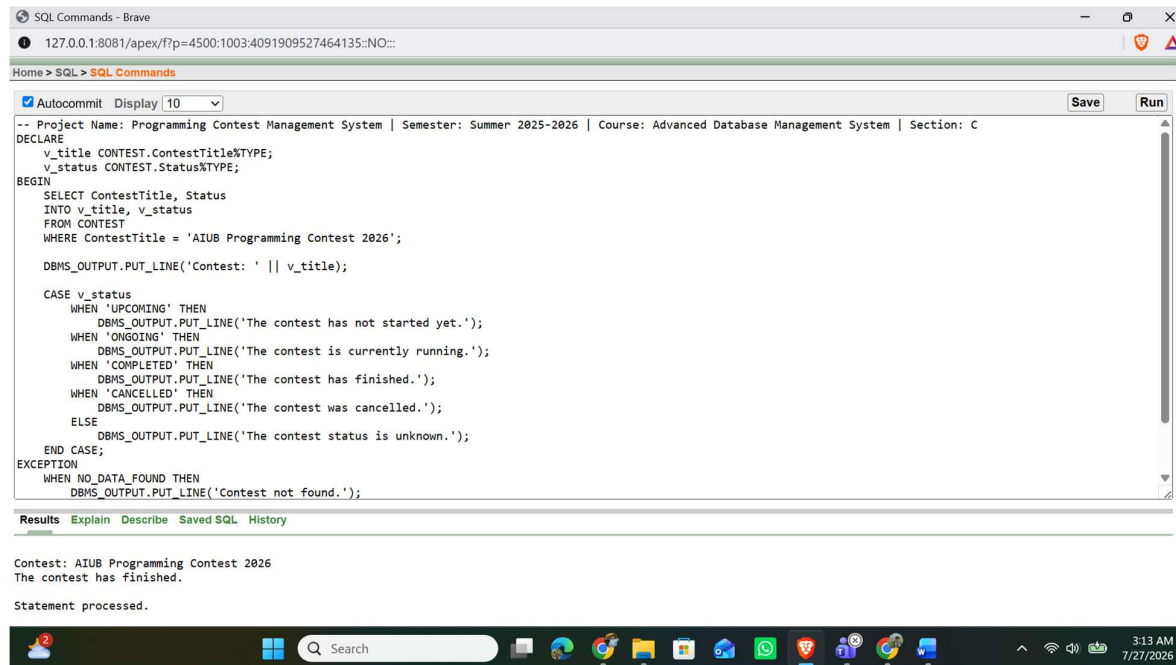
END CASE;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Contest not found.');
```

```

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one contest matched the title.');
```

```

END;
```



Category: Subqueries

Q13: Write a PL/SQL program that displays users whose ratings are above the average user rating.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

BEGIN
    FOR rec IN (
        SELECT UserName, Rating
        FROM CONTEST_USER
        WHERE NVL(Rating, 0) > (
            SELECT AVG(NVL(Rating, 0))
            FROM CONTEST_USER
        )
        ORDER BY Rating DESC
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            'User: ' || rec.UserName ||
            ' | Rating: ' || NVL(rec.Rating, 0)
        );
    END LOOP;
END;
```

SQL Commands - Brave

127.0.0.1:8081/apex/?p=4500:1003:4091909527464135::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

☒ Autocommit Display: 10 Save Run

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

BEGIN
  FOR rec IN (
    SELECT UserName, Rating
    FROM CONTEST_USER
    WHERE NVL(Rating, 0) > (
      SELECT AVG(NVL(Rating, 0))
      FROM CONTEST_USER
    )
    ORDER BY Rating DESC
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'User: ' || rec.UserName ||
      ' | Rating: ' || NVL(rec.Rating, 0)
    );
  END LOOP;
END;

```

Results Explain Describe Saved SQL History

User: Asad Bin Jafor | Rating: 2010
 User: MD Arman | Rating: 1940
 User: Roman Sheikh | Rating: 1850

Statement processed.

0.00 seconds

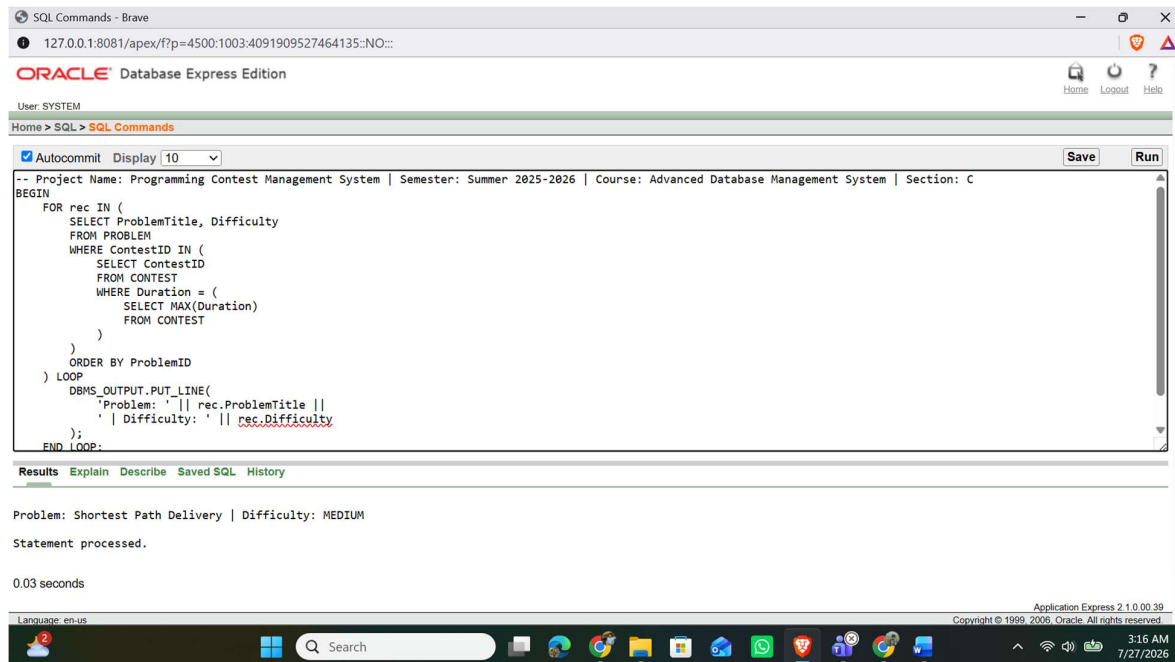
Q14: Write a PL/SQL program that displays problems belonging to the contest or contests with the longest duration.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

BEGIN
  FOR rec IN (
    SELECT ProblemTitle, Difficulty
    FROM PROBLEM
    WHERE ContestID IN (
      SELECT ContestID
      FROM CONTEST
      WHERE Duration = (
        SELECT MAX(Duration)
        FROM CONTEST
      )
    )
    ORDER BY ProblemID
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Problem: ' || rec.ProblemTitle ||
      ' | Difficulty: ' || rec.Difficulty
    );
  END LOOP;
END;

```



Category: Joins

Q15: Write a PL/SQL program that displays every contest title with its organizer name using an ANSI JOIN.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

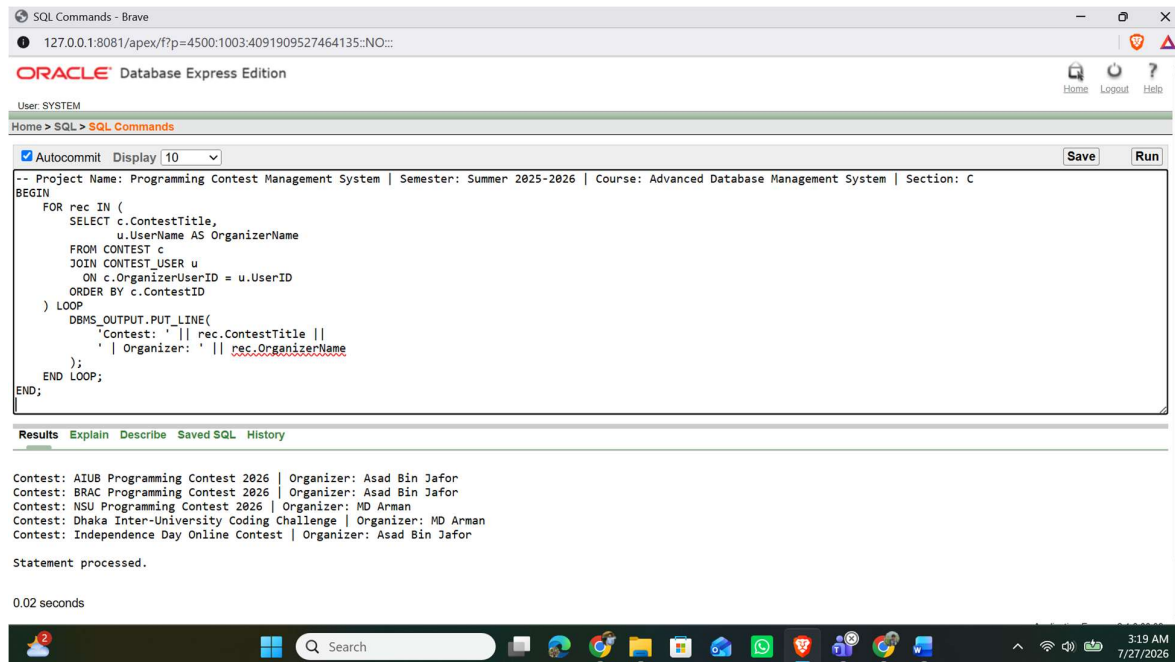
BEGIN

```

  FOR rec IN (
    SELECT c.ContestTitle,
           u.UserName AS OrganizerName
    FROM CONTEST c
    JOIN CONTEST_USER u
      ON c.OrganizerUserID = u.UserID
    ORDER BY c.ContestID
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Contest: ' || rec.ContestTitle ||
      ' | Organizer: ' || rec.OrganizerName
    );
  END LOOP;

```

END;



Q16: Write a PL/SQL program that displays submission IDs with participant names, problem titles, and verdicts using ANSI JOIN operations.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
 Advanced Database Management System | Section: C

```
BEGIN
  FOR rec IN (
    SELECT s.SubmissionID,
           u.UserName AS ParticipantName,
           p.ProblemTitle,
           s.Verdict
    FROM SUBMISSION s
    JOIN CONTEST_USER u
      ON s.UserID = u.UserID
    JOIN PROBLEM p
      ON s.ProblemID = p.ProblemID
    WHERE u.UserRole = 'PARTICIPANT'
    ORDER BY s.SubmissionID
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Submission ID: ' || rec.SubmissionID ||
      ' | Participant: ' || rec.ParticipantName ||
      ' | Problem: ' || rec.ProblemTitle ||
      ' | Verdict: ' || rec.Verdict
    );
  END LOOP;
END;
```

SQL Commands - Brave

127.0.0.1:8081/apex/?p=4500:1003:4091909527464135::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

☒ Autocommit Display: 10 Save Run

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

BEGIN
  FOR rec IN (
    SELECT s.SubmissionID,
           u.UserName AS ParticipantName,
           p.ProblemTitle,
           s.Verdict
    FROM SUBMISSION s
    JOIN CONTEST_USER u
      ON s.UserID = u.UserID
    JOIN PROBLEM p
      ON s.ProblemID = p.ProblemID
    WHERE u.UserRole = 'PARTICIPANT'
    ORDER BY s.SubmissionID
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Submission ID: ' || rec.SubmissionID ||
      ' | Participant: ' || rec.ParticipantName ||
      ' | Problem: ' || rec.ProblemTitle ||
      ' | Verdict: ' || rec.Verdict ||
    )
  END LOOP

```

Results Explain Describe Saved SQL History

Submission ID: 401	Participant: Taki Tajuar	Problem: Sum of Two Numbers	Verdict: ACCEPTED
Submission ID: 402	Participant: Taki Tajuar	Problem: Dynamic Sequence Queries	Verdict: TIME_LIMIT_EXCEEDED
Submission ID: 403	Participant: Shishir Mahmud	Problem: Balanced Brackets	Verdict: ACCEPTED
Submission ID: 404	Participant: Shishir Mahmud	Problem: Shortest Path Delivery	Verdict: WRONG_ANSWER
Submission ID: 405	Participant: Taki Tajuar	Problem: River Crossing Optimization	Verdict: PENDING

Statement processed.

3:20 AM 7/27/2026

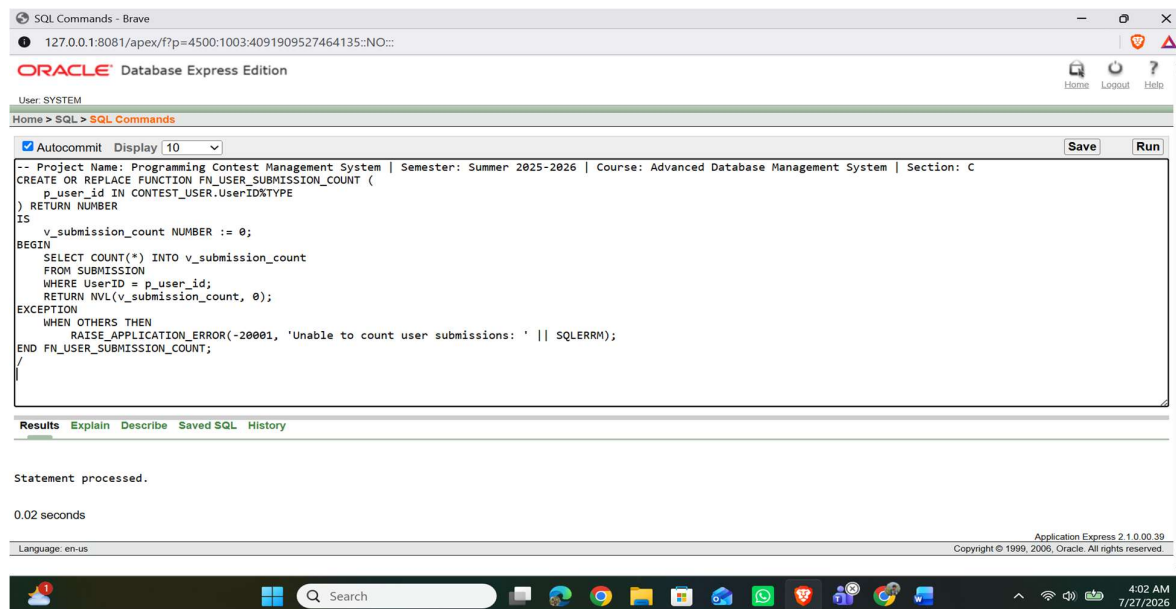
13.2 Advanced PL/SQL

Category: Stored Functions

Q1: Create a stored function that accepts a UserID and returns the total number of submissions made by that user.

CREATE FUNCTION :

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C
CREATE OR REPLACE FUNCTION FN_USER_SUBMISSION_COUNT (
    p_user_id IN CONTEST_USER.UserID%TYPE
) RETURN NUMBER
IS
    v_submission_count NUMBER := 0;
BEGIN
    SELECT COUNT(*) INTO v_submission_count
    FROM SUBMISSION
    WHERE UserID = p_user_id;
    RETURN NVL(v_submission_count, 0);
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20001, 'Unable to count user submissions: ' || SQLERRM);
END FN_USER_SUBMISSION_COUNT;
/
```



Calling the Function:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

DECLARE

```
v_user_id CONTEST_USER.UserID%TYPE;
```

```
v_count NUMBER;
```

```
BEGIN
```

```

SELECT UserID INTO v_user_id
FROM CONTEST_USER
WHERE Email = 'taki@gmail.com';

v_count := FN_USER_SUBMISSION_COUNT(v_user_id);

DBMS_OUTPUT.PUT_LINE('Submission count: ' || v_count);

EXCEPTION

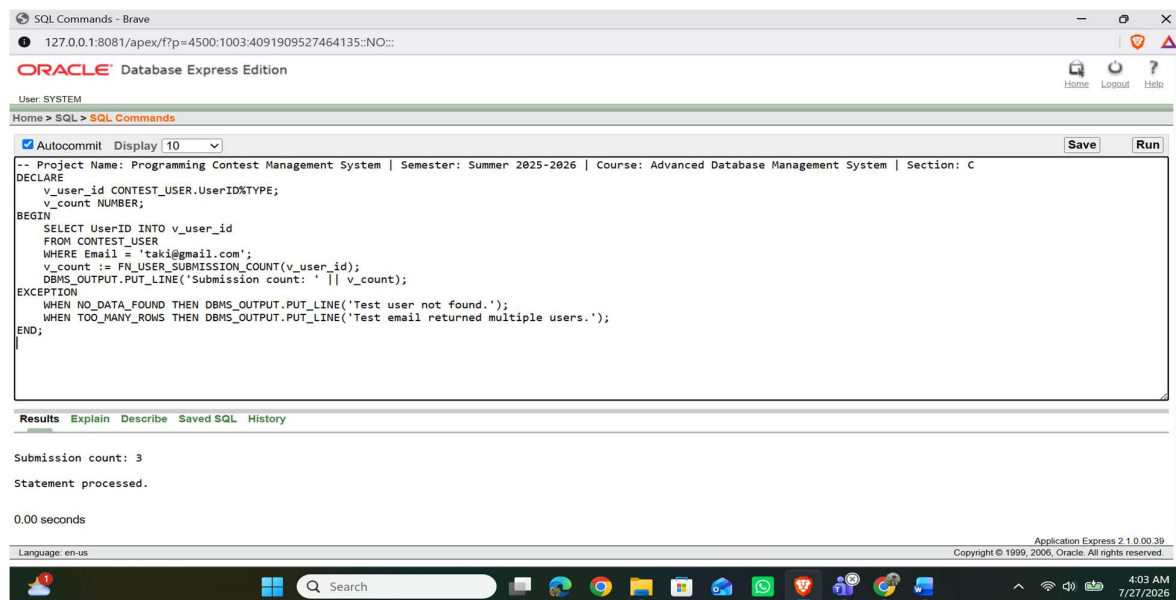
    WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Test user not found.');
```

```

    WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('Test email returned multiple users.');
```

```

END;
```



Q2: Create a stored function that accepts a ContestID, verifies the contest, and returns its total number of problems.

CREATE FUNCTION:

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C
```

```
CREATE OR REPLACE FUNCTION FN_CONTEST_PROBLEM_COUNT (
    p_contest_id IN CONTEST.ContestID%TYPE
) RETURN NUMBER
IS
```

```

    v_verified_id CONTEST.ContestID%TYPE;
    v_problem_count NUMBER := 0;
BEGIN
    SELECT ContestID INTO v_verified_id
    FROM CONTEST
    WHERE ContestID = p_contest_id;
    SELECT COUNT(*) INTO v_problem_count
    FROM PROBLEM
    WHERE ContestID = v_verified_id;
    RETURN NVL(v_problem_count, 0);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
```

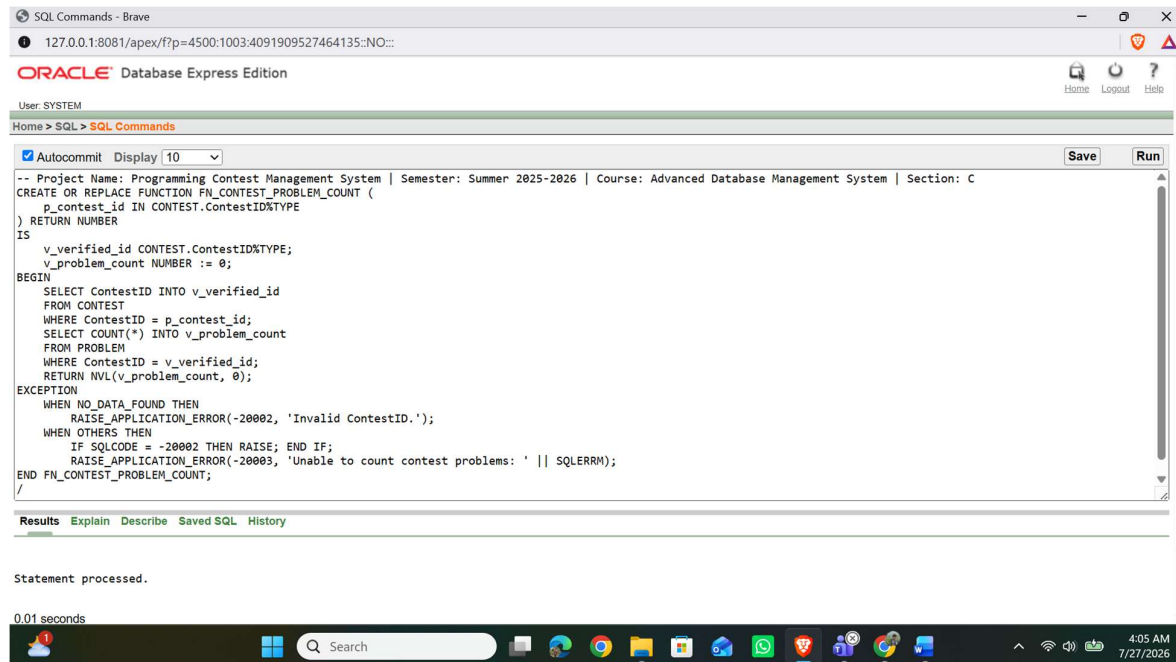
```

        RAISE_APPLICATION_ERROR(-20002, 'Invalid ContestID.');
```

WHEN OTHERS THEN

```

        IF SQLCODE = -20002 THEN RAISE; END IF;
        RAISE_APPLICATION_ERROR(-20003, 'Unable to count contest problems: ' || SQLERRM);
END FN_CONTEST_PROBLEM_COUNT;
/
```



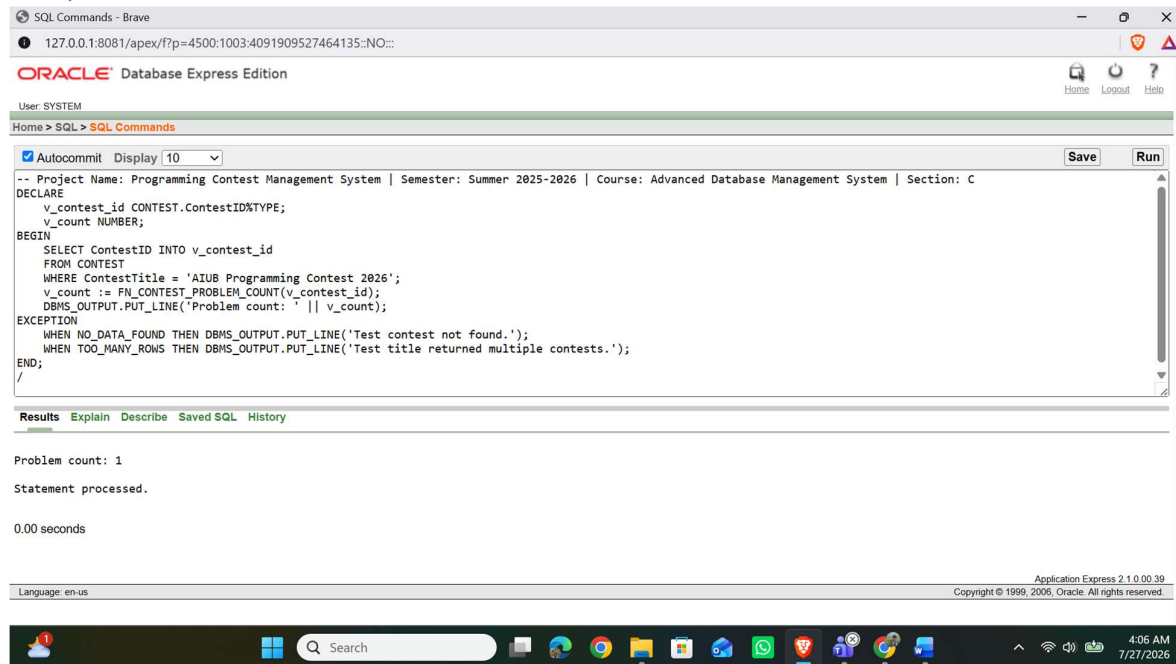
Calling the Function:

```

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C
DECLARE
    v_contest_id CONTEST.ContestID%TYPE;
    v_count NUMBER;
BEGIN
    SELECT ContestID INTO v_contest_id
    FROM CONTEST
    WHERE ContestTitle = 'AIUB Programming Contest 2026';
    v_count := FN_CONTEST_PROBLEM_COUNT(v_contest_id);
    DBMS_OUTPUT.PUT_LINE('Problem count: ' || v_count);
EXCEPTION
    WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Test contest not found.');
```

WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('Test title returned multiple contests.');

END;



Category: Stored Procedures

Q3: Create a procedure that validates a contest and inserts a new announcement using SEQ_ANNOUNCEMENT.NEXTVAL and SYSDATE.

CREATE PROCEDURE:

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C
CREATE OR REPLACE PROCEDURE SP_ADD_ANNOUNCEMENT (
    p_contest_id IN CONTEST.ContestID%TYPE,
    p_title IN ANNOUNCEMENT.AnnouncementTitle%TYPE,
    p_message IN ANNOUNCEMENT.Message%TYPE
)
IS
    v_contest_count NUMBER;
BEGIN
    IF p_title IS NULL OR p_message IS NULL THEN
        RAISE_APPLICATION_ERROR(-20010, 'Title and message are required.');
```

```
END IF;
    SELECT COUNT(*) INTO v_contest_count FROM CONTEST WHERE ContestID = p_contest_id;
    IF v_contest_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20011, 'Invalid ContestID.');
```

```
END IF;
    INSERT INTO ANNOUNCEMENT
        (AnnouncementID, AnnouncementTitle, Message, CreatedDate, ContestID)
    VALUES
        (SEQ_ANNOUNCEMENT.NEXTVAL, p_title, p_message, SYSDATE, p_contest_id);
    DBMS_OUTPUT.PUT_LINE('Announcement inserted.');
```

```
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE BETWEEN -20999 AND -20000 THEN RAISE; END IF;
        RAISE_APPLICATION_ERROR(-20012, 'Unable to add announcement: ' || SQLERRM);
```

```
END SP_ADD_ANNOUNCEMENT;
/
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
CREATE OR REPLACE PROCEDURE SP_ADD_ANNOUNCEMENT (
  p_contest_id IN CONTEST.ContestID%TYPE,
  p_title IN ANNOUNCEMENT.AnnouncementTitle%TYPE,
  p_message IN ANNOUNCEMENT.Message%TYPE
)
IS
  v_contest_count NUMBER;
BEGIN
  IF p_title IS NULL OR p_message IS NULL THEN
    RAISE_APPLICATION_ERROR(-20010, 'Title and message are required.');
```

The Results pane shows: 0 row(s) inserted. 0.03 seconds.

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C
```

```
DECLARE
  v_contest_id CONTEST.ContestID%TYPE;
BEGIN
  SELECT ContestID INTO v_contest_id
  FROM CONTEST
  WHERE ContestTitle = 'AIUB Programming Contest 2026';
  SP_ADD_ANNOUNCEMENT(
    v_contest_id,
    'Advanced PL/SQL Test Notice',
    'This temporary announcement verifies the stored procedure.'
  );
END;
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
DECLARE
  v_contest_id CONTEST.ContestID%TYPE;
BEGIN
  SELECT ContestID INTO v_contest_id
  FROM CONTEST
  WHERE ContestTitle = 'AIUB Programming Contest 2026';
  SP_ADD_ANNOUNCEMENT(
    v_contest_id,
    'Advanced PL/SQL Test Notice',
    'This temporary announcement verifies the stored procedure.'
  );
END;
```

The Results pane shows: Announcement inserted. Statement processed. 0.03 seconds.

SQL Commands - Brave
127.0.0.1:8081/apex/f?p=4500:1003:4091909527464135::NO::

ORACLE Database Express Edition

User: SYSTEM
Home > SQL > SQL Commands

Autocommit Display 10 Save Run

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

select * from ANNOUNCEMENT;

Results Explain Describe Saved SQL History

ANNOUNCEMENTID	ANNOUNCEMENTTITLE	MESSAGE	CREATEDDATE	CONTESTID
601	Contest Registration Opened	Registration is now open for all participants.	01-JUL-26	101
602	Rule Update	Penalty will be calculated for accepted problems.	22-JUL-26	102
603	Contest Starting Soon	Participants should join thirty minutes early.	30-JUL-26	103
604	Team List Published	The final list of participating teams has been published.	01-AUG-26	104
605	Final Result Published	The final contest result has been published.	27-MAR-26	105
606	Advanced PU/SQL Test Notice	This temporary announcement verifies the stored procedure.	27-JUL-26	101

6 rows returned in 0.02 seconds CSV Export

Language: en-us Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved. 4:24 AM 7/21/2026

Q4: Create a procedure that records a non-null clarification answer, changes the status to ANSWERED, and validates the affected row with SQL%ROWCOUNT.

CREATE PROCEDURE:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```
CREATE OR REPLACE PROCEDURE SP_ANSWER_CLARIFICATION (
    p_clarification_id IN CLARIFICATION.ClarificationID%TYPE,
    p_answer IN CLARIFICATION.Answer%TYPE
)
IS
BEGIN
    IF p_answer IS NULL THEN
        RAISE_APPLICATION_ERROR(-20020, 'Answer text cannot be NULL.');
```

END IF;

```
    UPDATE CLARIFICATION
    SET Answer = p_answer, Status = 'ANSWERED'
    WHERE ClarificationID = p_clarification_id;
    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20021, 'Invalid ClarificationID.');
```

END IF;

```
    DBMS_OUTPUT.PUT_LINE('Clarification answered.');
```

```
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE BETWEEN -20999 AND -20000 THEN RAISE; END IF;
        RAISE_APPLICATION_ERROR(-20022, 'Unable to answer clarification: ' || SQLERRM);
END SP_ANSWER_CLARIFICATION;
/
```

SQL Commands - Brave
127.0.0.1:8081/apex/?p=4500:1003:4091909527464135::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
CREATE OR REPLACE PROCEDURE SP_ANSWER_CLARIFICATION (
    p_clarification_id IN CLARIFICATION.ClarificationID%TYPE,
    p_answer IN CLARIFICATION.Answer%TYPE
)
IS
BEGIN
    IF p_answer IS NULL THEN
        RAISE_APPLICATION_ERROR(-20020, 'Answer text cannot be NULL.');
```

Results Explain Describe Saved SQL History

0 row(s) updated.

0.02 seconds

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

DECLARE

 v_clarification_id CLARIFICATION.ClarificationID%TYPE;

BEGIN

 SELECT MIN(ClarificationID) INTO v_clarification_id

 FROM CLARIFICATION

 WHERE Status = 'OPEN';

 IF v_clarification_id IS NULL THEN

 RAISE_APPLICATION_ERROR(-20023, 'No OPEN clarification is available.');

 END IF;

 SP_ANSWER_CLARIFICATION(

 v_clarification_id,

 'Temporary answer created for procedure verification.'

);

END;

SQL Commands - Brave
127.0.0.1:8081/apex/?p=4500:1003:4091909527464135::NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
DECLARE
    v_clarification_id CLARIFICATION.ClarificationID%TYPE;
BEGIN
    SELECT MIN(ClarificationID) INTO v_clarification_id
    FROM CLARIFICATION
    WHERE Status = 'OPEN';
    IF v_clarification_id IS NULL THEN
        RAISE_APPLICATION_ERROR(-20023, 'No OPEN clarification is available.');
```

Results Explain Describe Saved SQL History

Clarification answered.

Statement processed.

0.01 seconds

Language: en-us

Application Express 2.1.0.0.39
Copyright © 1999, 2006, Oracle. All rights reserved.

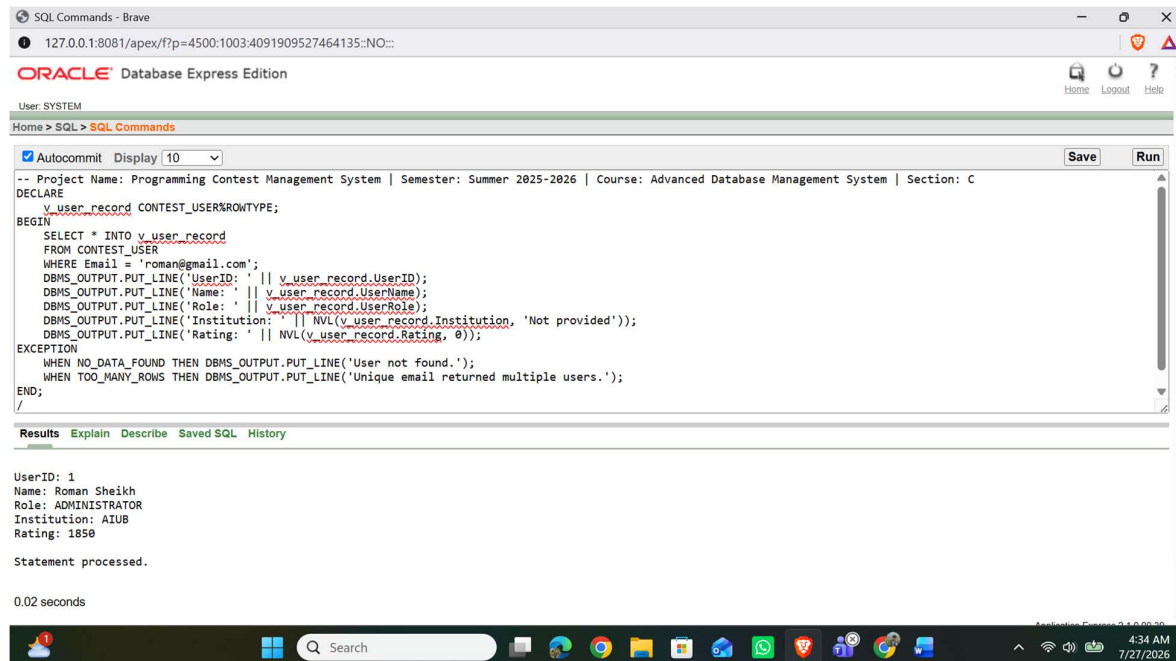
Category: Table-Based Records

Q5: Use a table-based record to retrieve one user by unique email and display selected non-sensitive fields.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C

```
DECLARE
    v_user_record CONTEST_USER%ROWTYPE;
BEGIN
    SELECT * INTO v_user_record
    FROM CONTEST_USER
    WHERE Email = 'ayesha.sultana@bracu.ac.bd';
    DBMS_OUTPUT.PUT_LINE('UserID: ' || v_user_record.UserID);
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_user_record.UserName);
    DBMS_OUTPUT.PUT_LINE('Role: ' || v_user_record.UserRole);
    DBMS_OUTPUT.PUT_LINE('Institution: ' || NVL(v_user_record.Institution, 'Not provided'));
    DBMS_OUTPUT.PUT_LINE('Rating: ' || NVL(v_user_record.Rating, 0));
EXCEPTION
    WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('User not found.');
```

END;



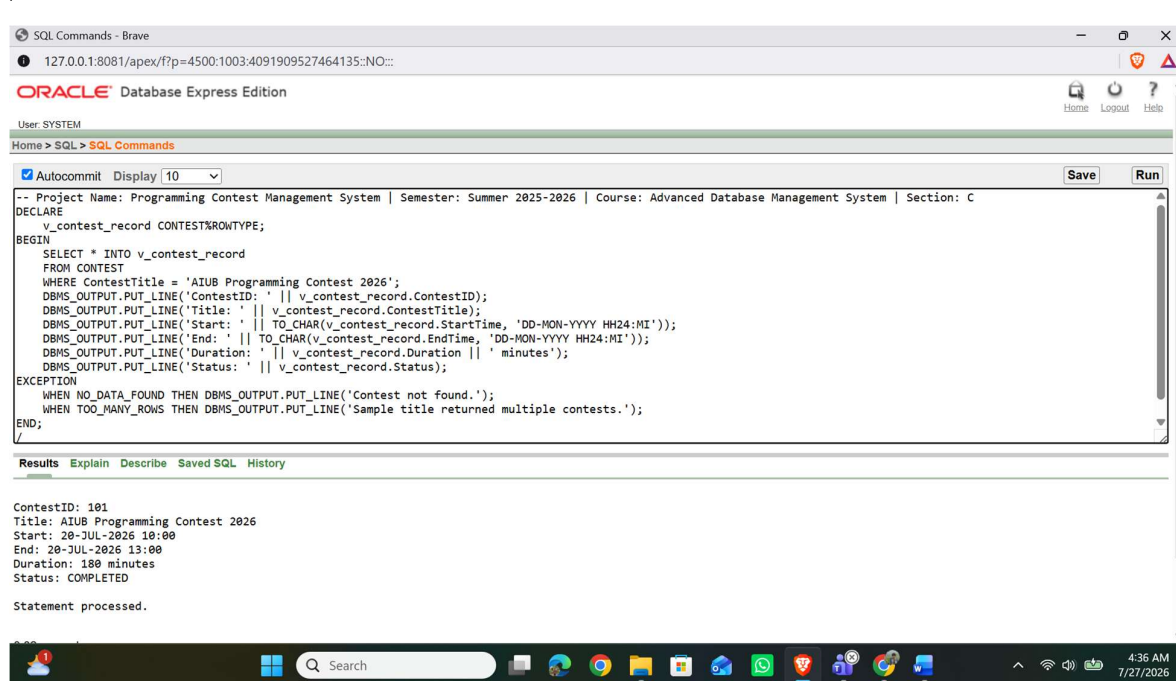
Q6: Use a table-based record to retrieve one contest by unique sample title and display its key scheduling fields.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C

```
DECLARE
    v_contest_record CONTEST%ROWTYPE;
BEGIN
    SELECT * INTO v_contest_record
    FROM CONTEST
    WHERE ContestTitle = 'AIUB Programming Contest 2026';
```

```

DBMS_OUTPUT.PUT_LINE('ContestID: ' || v_contest_record.ContestID);
DBMS_OUTPUT.PUT_LINE('Title: ' || v_contest_record.ContestTitle);
DBMS_OUTPUT.PUT_LINE('Start: ' || TO_CHAR(v_contest_record.StartTime, 'DD-MON-YYYY
HH24:MI'));
DBMS_OUTPUT.PUT_LINE('End: ' || TO_CHAR(v_contest_record.EndTime, 'DD-MON-YYYY
HH24:MI'));
DBMS_OUTPUT.PUT_LINE('Duration: ' || v_contest_record.Duration || ' minutes');
DBMS_OUTPUT.PUT_LINE('Status: ' || v_contest_record.Status);
EXCEPTION
    WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Contest not found.');
```



Category: Explicit Cursors

Q7: Use a parameterized explicit cursor with separate FETCH variables to display every problem of a selected contest.

```

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C
DECLARE
```

```

    CURSOR c_problems (p_contest_id IN CONTEST.ContestID%TYPE) IS
        SELECT ProblemID, ProblemTitle, Difficulty, TimeLimit, MemoryLimit
        FROM PROBLEM
        WHERE ContestID = p_contest_id
        ORDER BY ProblemID;
    v_contest_id CONTEST.ContestID%TYPE;
    v_problem_id PROBLEM.ProblemID%TYPE;
    v_title PROBLEM.ProblemTitle%TYPE;
    v_difficulty PROBLEM.Difficulty%TYPE;
    v_time_limit PROBLEM.TimeLimit%TYPE;
    v_memory_limit PROBLEM.MemoryLimit%TYPE;
```

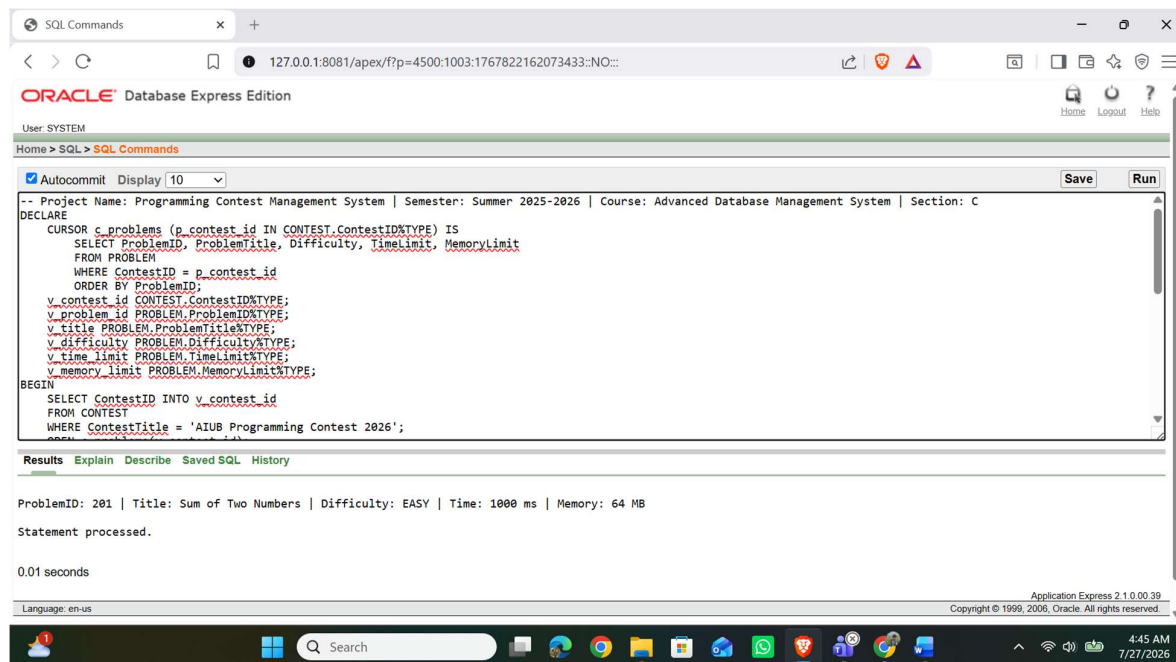
```

BEGIN
  SELECT ContestID INTO v_contest_id
  FROM CONTEST
  WHERE ContestTitle = 'AIUB Programming Contest 2026';
  OPEN c_problems(v_contest_id);
  LOOP
    FETCH c_problems
    INTO v_problem_id, v_title, v_difficulty, v_time_limit, v_memory_limit;
    EXIT WHEN c_problems%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(
      'ProblemID: ' || v_problem_id ||
      ' | Title: ' || v_title ||
      ' | Difficulty: ' || v_difficulty ||
      ' | Time: ' || v_time_limit || ' ms' ||
      ' | Memory: ' || v_memory_limit || ' MB'
    );
  END LOOP;
  CLOSE c_problems;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    IF c_problems%ISOPEN THEN CLOSE c_problems; END IF;
    DBMS_OUTPUT.PUT_LINE('Selected contest not found.');
```

WHEN TOO_MANY_ROWS THEN
 IF c_problems%ISOPEN THEN CLOSE c_problems; END IF;
 DBMS_OUTPUT.PUT_LINE('Selected title returned multiple contests.');

```

END;
/
```



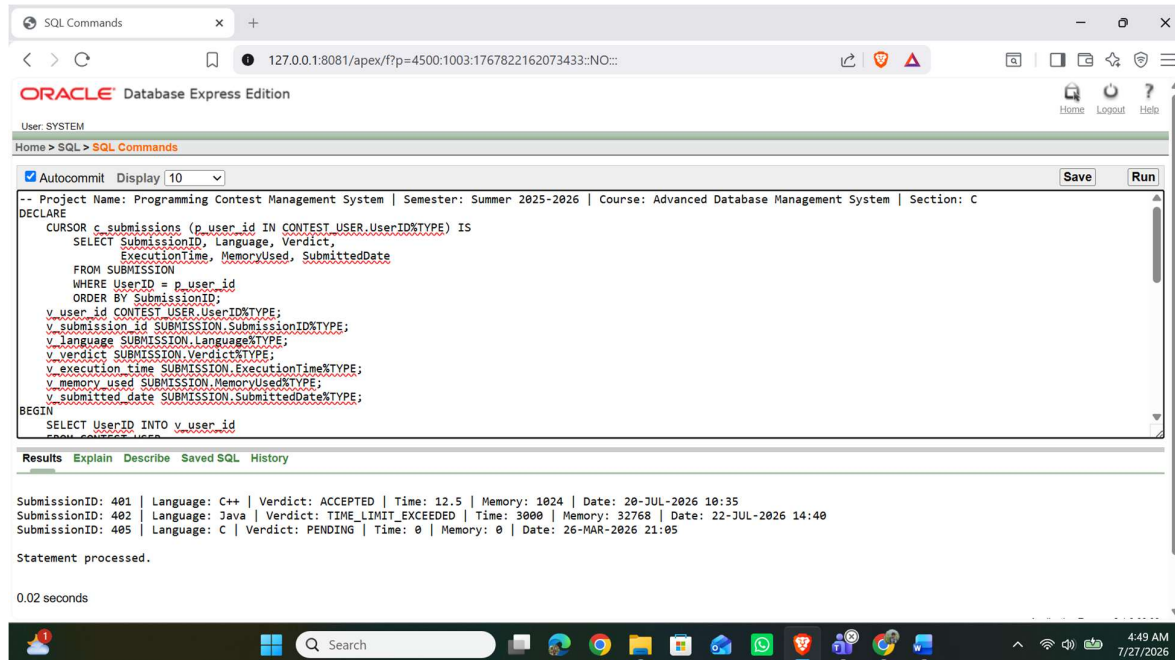
Q8: Use an explicit cursor and separate scalar FETCH variables to display every submission made by a selected participant.

Answer:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C

```
DECLARE
CURSOR c_submissions (p_user_id IN CONTEST_USER.UserID%TYPE) IS
    SELECT SubmissionID, Language, Verdict,
           ExecutionTime, MemoryUsed, SubmittedDate
    FROM SUBMISSION
    WHERE UserID = p_user_id
    ORDER BY SubmissionID;
v_user_id CONTEST_USER.UserID%TYPE;
v_submission_id SUBMISSION.SubmissionID%TYPE;
v_language SUBMISSION.Language%TYPE;
v_verdict SUBMISSION.Verdict%TYPE;
v_execution_time SUBMISSION.ExecutionTime%TYPE;
v_memory_used SUBMISSION.MemoryUsed%TYPE;
v_submitted_date SUBMISSION.SubmittedDate%TYPE;
BEGIN
    SELECT UserID INTO v_user_id
    FROM CONTEST_USER
    WHERE Email = 'ayesha.sultana@bracu.ac.bd'
    AND UserRole = 'PARTICIPANT';
    OPEN c_submissions(v_user_id);
    LOOP
        FETCH c_submissions
        INTO v_submission_id, v_language, v_verdict,
            v_execution_time, v_memory_used, v_submitted_date;
        EXIT WHEN c_submissions%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(
            'SubmissionID: ' || v_submission_id ||
            ' | Language: ' || v_language ||
            ' | Verdict: ' || v_verdict ||
            ' | Time: ' || NVL(v_execution_time, 0) ||
            ' | Memory: ' || NVL(v_memory_used, 0) ||
            ' | Date: ' || TO_CHAR(v_submitted_date, 'DD-MON-YYYY HH24:MI')
        );
    END LOOP;
    CLOSE c_submissions;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        IF c_submissions%ISOPEN THEN CLOSE c_submissions; END IF;
        DBMS_OUTPUT.PUT_LINE('Selected participant not found. ');
    WHEN TOO_MANY_ROWS THEN
        IF c_submissions%ISOPEN THEN CLOSE c_submissions; END IF;
        DBMS_OUTPUT.PUT_LINE('Selected email returned multiple users. ');
END;
```

/



Category: Cursor-Based Records

Q9: Use a cursor-based %ROWTYPE record to display contest titles, organizer names, and statuses.

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

DECLARE
    CURSOR c_contest_organizers IS
        SELECT c.ContestTitle,
               u.UserName AS OrganizerName,
               c.Status
        FROM CONTEST c
        JOIN CONTEST_USER u
          ON c.OrganizerUserID = u.UserID
        ORDER BY c.ContestID;
    v_contest_record c_contest_organizers%ROWTYPE;
BEGIN
    OPEN c_contest_organizers;
    LOOP
        FETCH c_contest_organizers INTO v_contest_record;
        EXIT WHEN c_contest_organizers%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(
            'Contest: ' || v_contest_record.ContestTitle ||
            '| Organizer: ' || v_contest_record.OrganizerName ||
            '| Status: ' || v_contest_record.Status
        );
    END LOOP;
    CLOSE c_contest_organizers;
EXCEPTION
    WHEN OTHERS THEN
        IF c_contest_organizers%ISOPEN THEN CLOSE c_contest_organizers; END IF;
        DBMS_OUTPUT.PUT_LINE('Cursor error: ' || SQLERRM);
END;
/

```



```

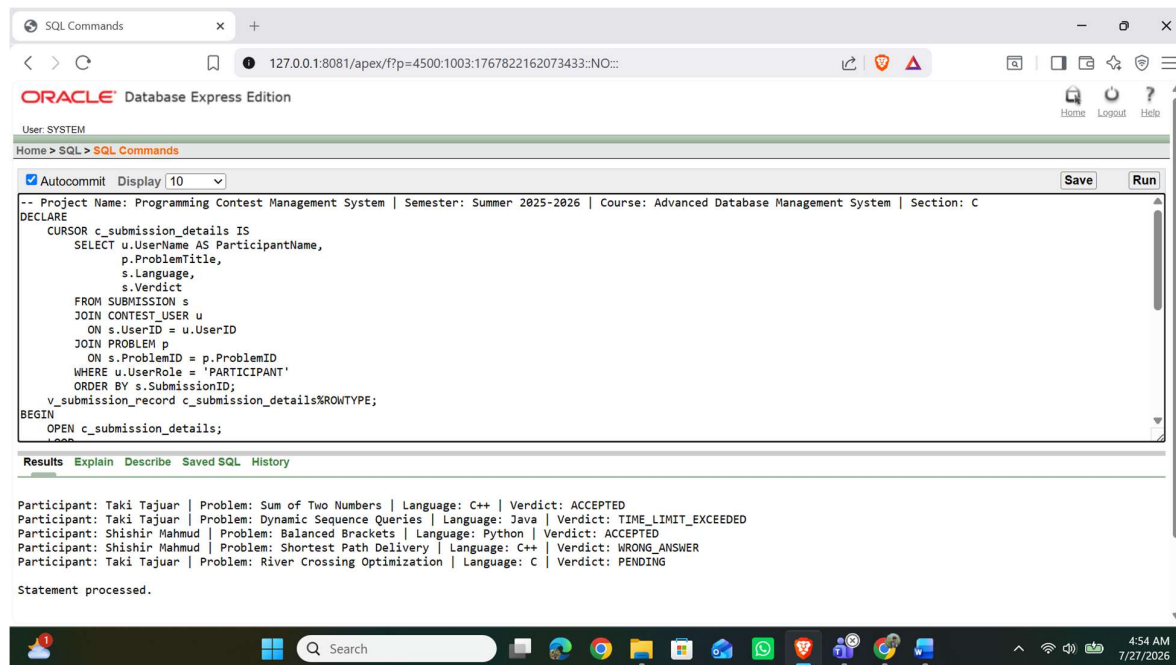
DECLARE
CURSOR c_submission_details IS
    SELECT u.UserName AS ParticipantName,
           p.ProblemTitle,
           s.Language,
           s.Verdict
    FROM SUBMISSION s
    JOIN CONTEST_USER u
        ON s.UserID = u.UserID
    JOIN PROBLEM p
        ON s.ProblemID = p.ProblemID
    WHERE u.UserRole = 'PARTICIPANT'
    ORDER BY s.SubmissionID;
v_submission_record c_submission_details%ROWTYPE;
BEGIN
    OPEN c_submission_details;
    LOOP
        FETCH c_submission_details INTO v_submission_record;
        EXIT WHEN c_submission_details%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(
            'Participant: ' || v_submission_record.ParticipantName ||
            ' | Problem: ' || v_submission_record.ProblemTitle ||
            ' | Language: ' || v_submission_record.Language ||
            ' | Verdict: ' || v_submission_record.Verdict
        );
    END LOOP;
    CLOSE c_submission_details;
EXCEPTION

```

```

WHEN OTHERS THEN
    IF c_submission_details%ISOPEN THEN CLOSE c_submission_details; END IF;
    DBMS_OUTPUT.PUT_LINE('Cursor error: ' || SQLERRM);
END;
/

```



Category: Row-Level Triggers

Q11: Write a row-level trigger that displays the participant ID, previous rating, new rating, and rating difference whenever a participant's rating is updated.

Create Trigger:

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:
Advanced Database Management System | Section: C
```

```
CREATE OR REPLACE TRIGGER TRG_PARTICIPANT_RATING_CHANGE
```

```
AFTER UPDATE OF Rating ON CONTEST_USER
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Participant ID: ' || :OLD.UserID);
```

```
    DBMS_OUTPUT.PUT_LINE('Previous rating: ' || :OLD.Rating);
```

```
    DBMS_OUTPUT.PUT_LINE('New rating: ' || :NEW.Rating);
```

```
    DBMS_OUTPUT.PUT_LINE(
        'Rating difference: ' || (:NEW.Rating - :OLD.Rating) );
```

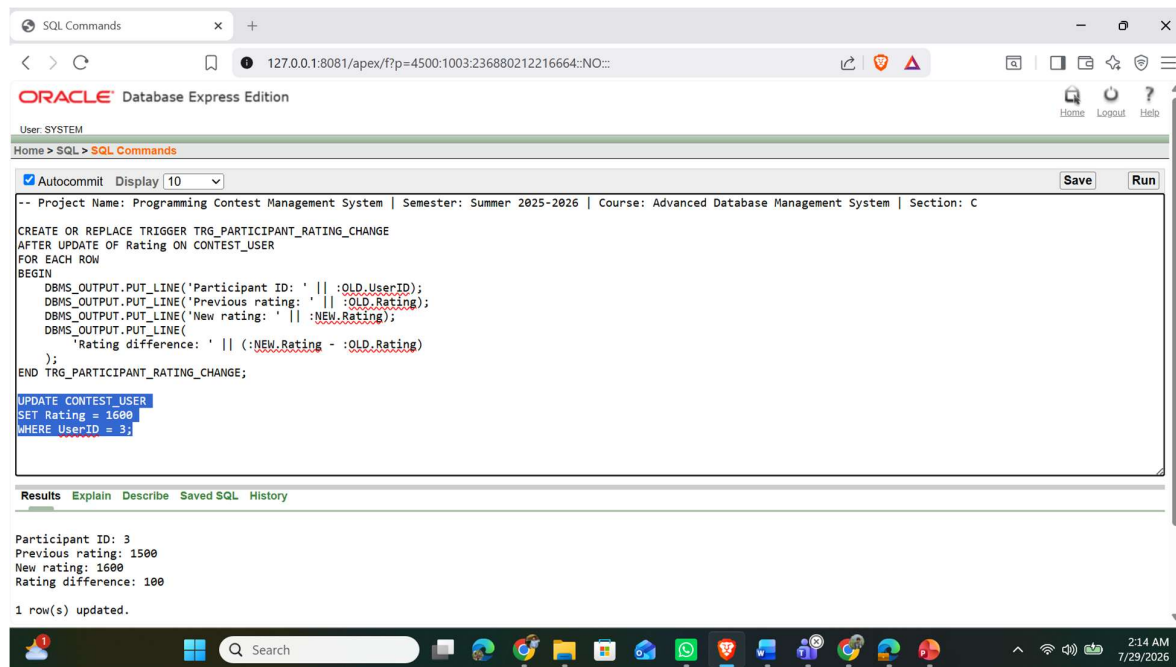
```
END TRG_PARTICIPANT_RATING_CHANGE;
```

Test Trigger:

```
UPDATE CONTEST_USER
```

```
SET Rating = 1600
```

WHERE UserID = 3;



Q12: Write a row-level trigger that displays the submission ID, participant ID, problem ID, language, and verdict whenever a new submission is inserted.

Create Trigger:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

CREATE OR REPLACE TRIGGER TRG_NEW_SUBMISSION_ROW

AFTER INSERT ON SUBMISSION

FOR EACH ROW

BEGIN

DBMS_OUTPUT.PUT_LINE('New submission inserted.');

DBMS_OUTPUT.PUT_LINE('Submission ID: ' || :NEW.SubmissionID);

DBMS_OUTPUT.PUT_LINE('Participant ID: ' || :NEW.UserID);

DBMS_OUTPUT.PUT_LINE('Problem ID: ' || :NEW.ProblemID);

DBMS_OUTPUT.PUT_LINE('Language: ' || :NEW.Language);

DBMS_OUTPUT.PUT_LINE ('Verdict: ' || NVL(:NEW.Verdict, 'NOT SET'));

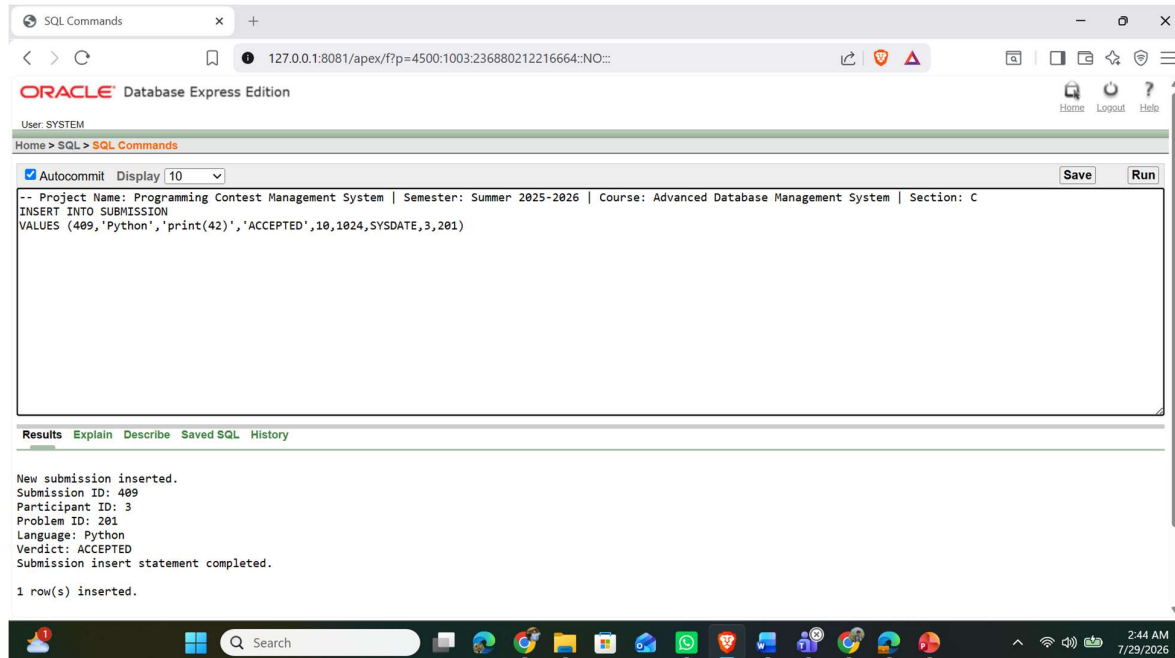
END TRG_NEW_SUBMISSION_ROW;

Test Trigger:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

INSERT INTO SUBMISSION

VALUES (409, 'Python', 'print(42)', 'ACCEPTED', 10, 1024, SYSDATE, 3, 201)



Category: Statement-Level Triggers

Q13: Write a statement-level trigger that displays a message after a SUBMISSION insert statement completes.

Create Trigger:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

CREATE OR REPLACE TRIGGER TRG_SUBMISSION_STATEMENT

AFTER INSERT ON SUBMISSION

BEGIN

DBMS_OUTPUT.PUT_LINE ('Submission insert statement completed.');

END TRG_SUBMISSION_STATEMENT;

Test Trigger:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

INSERT INTO SUBMISSION

(SubmissionID, Language, SourceCode, Verdict,

ExecutionTime, MemoryUsed, SubmittedDate,

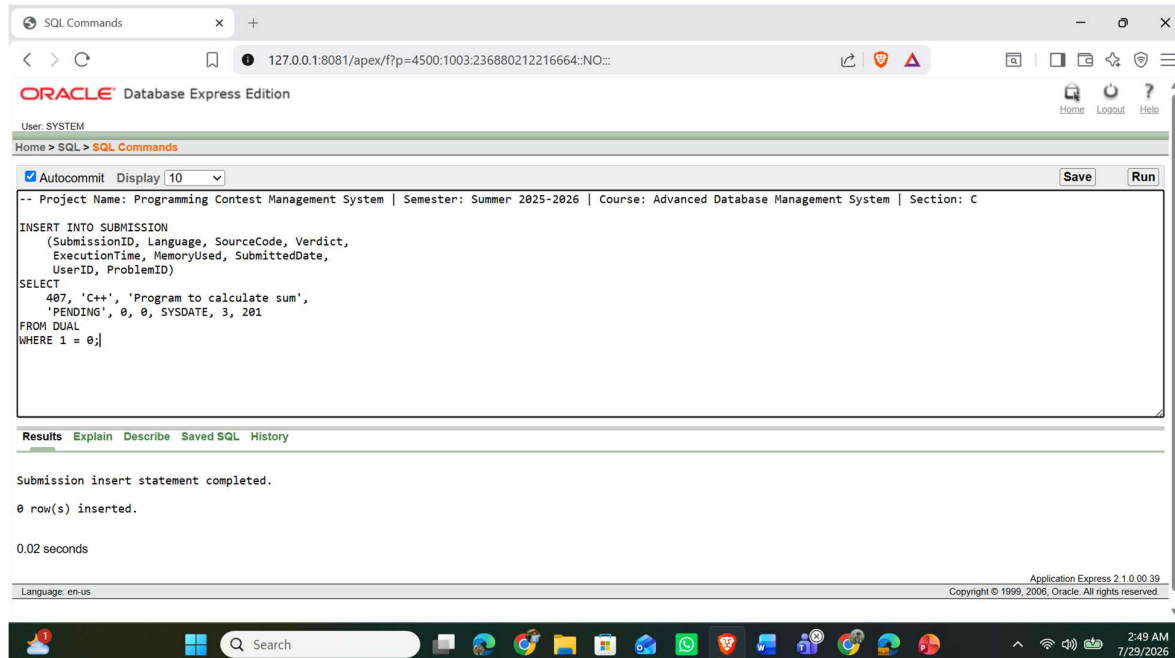
UserID, ProblemID)

SELECT

407, 'C++', 'Program to calculate sum',

'PENDING', 0, 0, SYSDATE, 3, 201

FROM DUAL WHERE 1 = 0;



Q14: Write a statement-level trigger that displays a message after an update statement changes the Status column of the CONTEST table.

Create Trigger:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

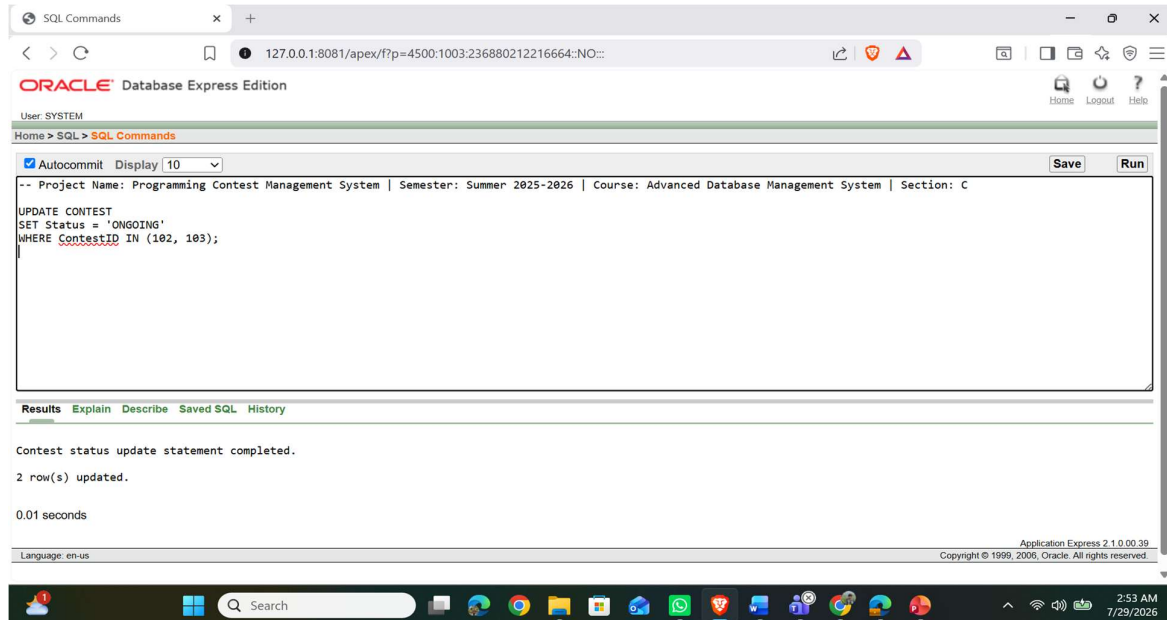
```
CREATE OR REPLACE TRIGGER TRG_CONTEST_STATUS_STATEMENT
AFTER UPDATE OF Status
ON CONTEST
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE( 'Contest status update statement completed.' );
END TRG_CONTEST_STATUS_STATEMENT;
```

Test Trigger:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```
UPDATE CONTEST
SET Status = 'ONGOING'
WHERE ContestID IN (102, 103);
```



Category: Packages

Q15: Write a PL/SQL package containing two procedures. The first procedure displays a user's name, and the second procedure displays the user's rating using the user ID.

Package Specification

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

CREATE OR REPLACE PACKAGE USER_PACK AS

PROCEDURE DISPLAY_USER_NAME(p_user_id CONTEST_USER.UserID%TYPE); PROCEDURE
DISPLAY_USER_RATING(p_user_id CONTEST_USER.UserID%TYPE);

END USER_PACK;

Package Body:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

CREATE OR REPLACE PACKAGE BODY USER_PACK AS

PROCEDURE DISPLAY_USER_NAME(p_user_id CONTEST_USER.UserID%TYPE)

IS

 v_user_name CONTEST_USER.UserName%TYPE;

BEGIN

 SELECT UserName INTO v_user_name

 FROM CONTEST_USER

 WHERE UserID = p_user_id;

 DBMS_OUTPUT.PUT_LINE('User Name: ' || v_user_name);

END DISPLAY_USER_NAME;

PROCEDURE DISPLAY_USER_RATING(p_user_id CONTEST_USER.UserID%TYPE)

IS

```

        v_user_rating CONTEST_USER.Rating%%TYPE;
BEGIN
    SELECT Rating INTO v_user_rating
    FROM CONTEST_USER
    WHERE UserID = p_user_id;
    DBMS_OUTPUT.PUT_LINE( 'User Rating: ' || v_user_rating );
END DISPLAY_USER_RATING;
END USER_PACK;

```

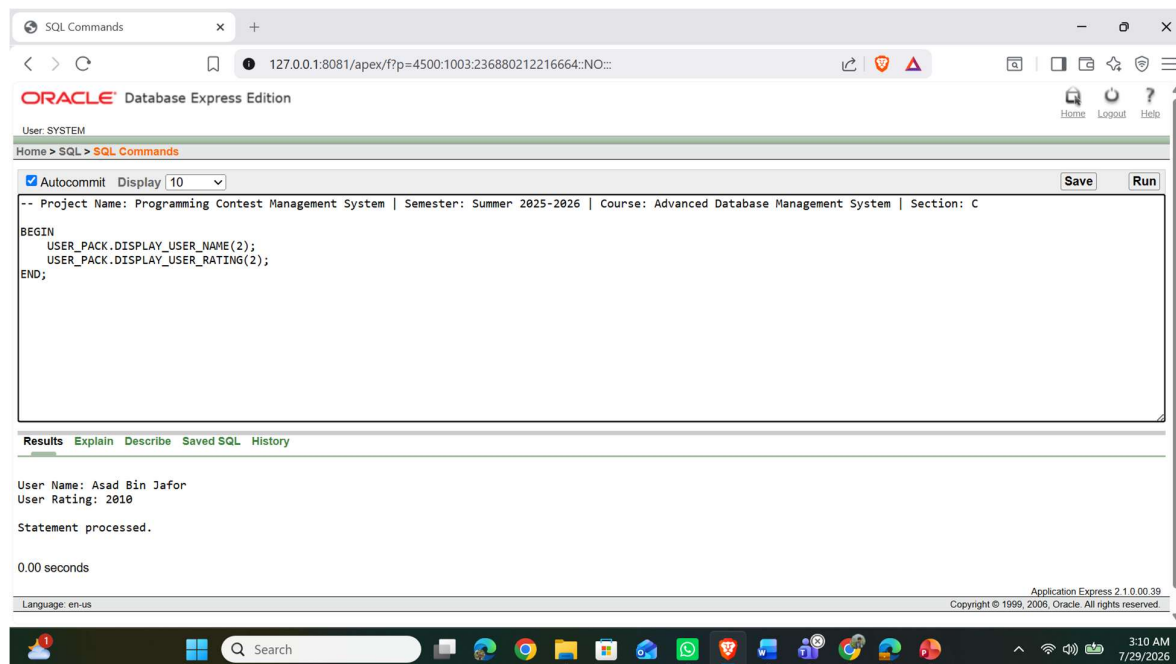
Execute the Package:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

BEGIN
    USER_PACK.DISPLAY_USER_NAME(2);
    USER_PACK.DISPLAY_USER_RATING(2);
END;

```



Q16: Write a PL/SQL package containing two procedures. The first procedure displays a problem title, and the second procedure displays the problem difficulty using the problem ID.

Package Specification:

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C

```

CREATE OR REPLACE PACKAGE PROBLEM_PACK AS
    PROCEDURE DISPLAY_PROBLEM_TITLE( p_problem_id PROBLEM.ProblemID%%TYPE );
    PROCEDURE DISPLAY_PROBLEM_DIFFICULTY( p_problem_id PROBLEM.ProblemID%%TYPE );

```

```
END PROBLEM_PACK;
```

Package Body:

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:  
Advanced Database Management System | Section: C
```

```
CREATE OR REPLACE PACKAGE BODY PROBLEM_PACK AS
```

```
    PROCEDURE DISPLAY_PROBLEM_TITLE( p_problem_id PROBLEM.ProblemID%TYPE )
```

```
    IS
```

```
        v_problem_title PROBLEM.ProblemTitle%TYPE;
```

```
    BEGIN
```

```
        SELECT ProblemTitle INTO v_problem_title
```

```
        FROM PROBLEM
```

```
        WHERE ProblemID = p_problem_id;
```

```
        DBMS_OUTPUT.PUT_LINE( 'Problem Title: ' || v_problem_title );
```

```
    END DISPLAY_PROBLEM_TITLE;
```

```
    PROCEDURE DISPLAY_PROBLEM_DIFFICULTY( p_problem_id PROBLEM.ProblemID%TYPE )
```

```
    IS
```

```
        v_difficulty PROBLEM.Difficulty%TYPE;
```

```
    BEGIN
```

```
        SELECT Difficulty INTO v_difficulty
```

```
        FROM PROBLEM
```

```
        WHERE ProblemID = p_problem_id;
```

```
        DBMS_OUTPUT.PUT_LINE( 'Problem Difficulty: ' || v_difficulty );
```

```
    END DISPLAY_PROBLEM_DIFFICULTY;
```

```
END PROBLEM_PACK;
```

Execute the Package:

```
-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course:  
Advanced Database Management System | Section: C
```

```
BEGIN
```

```
    PROBLEM_PACK.DISPLAY_PROBLEM_TITLE(201);
```

```
    PROBLEM_PACK.DISPLAY_PROBLEM_DIFFICULTY(201);
```

```
END;
```

SQL Commands

127.0.0.1:8081/apex/f?p=4500:1003:236880212216664:NO::

ORACLE Database Express Edition

User: SYSTEM

Home > SQL > SQL Commands

☒ Autocommit Display 10

Save Run

-- Project Name: Programming Contest Management System | Semester: Summer 2025-2026 | Course: Advanced Database Management System | Section: C
BEGIN
 PROBLEM_PACK.DISPLAY_PROBLEM_TITLE(201);
 PROBLEM_PACK.DISPLAY_PROBLEM_DIFFICULTY(201);
END;
/

Results Explain Describe Saved SQL History

Problem Title: Sum of Two Numbers
Problem Difficulty: EASY

Statement processed.

0.00 seconds

Language: en-us Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

81°

Search

3:19 AM
7/29/2026

14. Conclusion

The Programming Contest Management System makes programming contests easier to organize and manage. It keeps all important information in one place. The system handles users, contests, problems, test cases, submissions, announcements, clarifications, and leaderboards. It also helps reduce errors and manual work. Important rules are applied automatically through SQL and PL/SQL. This makes the contest process more accurate, fair, and reliable.

Overall, PCMS provides a simple and organized way to manage a programming contest from registration to final ranking.